컴퓨터구조

Presented by
오 세 춘

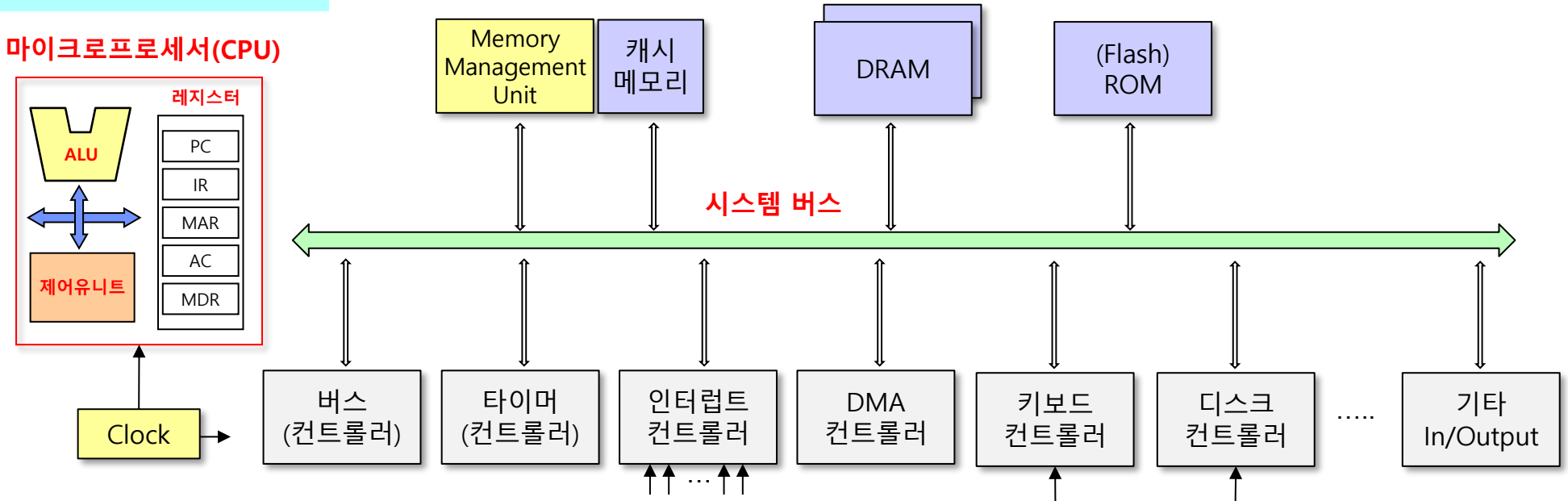
3장 컴퓨터 산술과 논리 연산(ALU)

교재 주요 구성

■ 컴퓨터 시스템의 구성 – 교재 및 강의 구성 내용

제1장 : 컴퓨터시스템 개요

제5장 : 기억장치



제2장 : CPU의 구조와 기능

제3장 : 컴퓨터 산술과 논리 연산(ALU)

제4장 : 제어 유니트

제7장 : 시스템 버스와 IO 및 인터럽트

추가: 가상메모리

제6장 : 보조기억장치

제8장 : 고성능 컴퓨터시스템 구조

배울 내용

■ 목차

✓ 3.1 ALU의 구성 요소

✓ 3.2 정수의 표현

✓ 3.3 논리 연산

✓ 3.4 시프트 연산

✓ 3.5 정수의 산술 연산

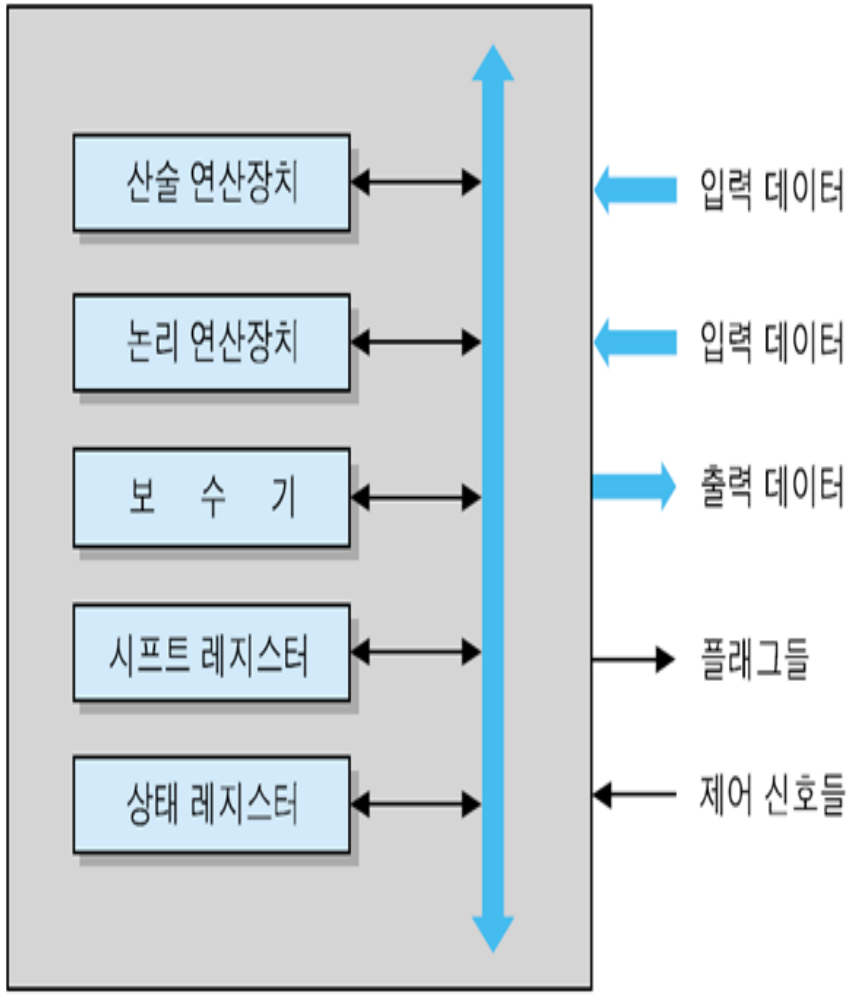
✓ 3.6 부동소수점 수의 표현

✓ 3.7 부동소수점 산술 연산

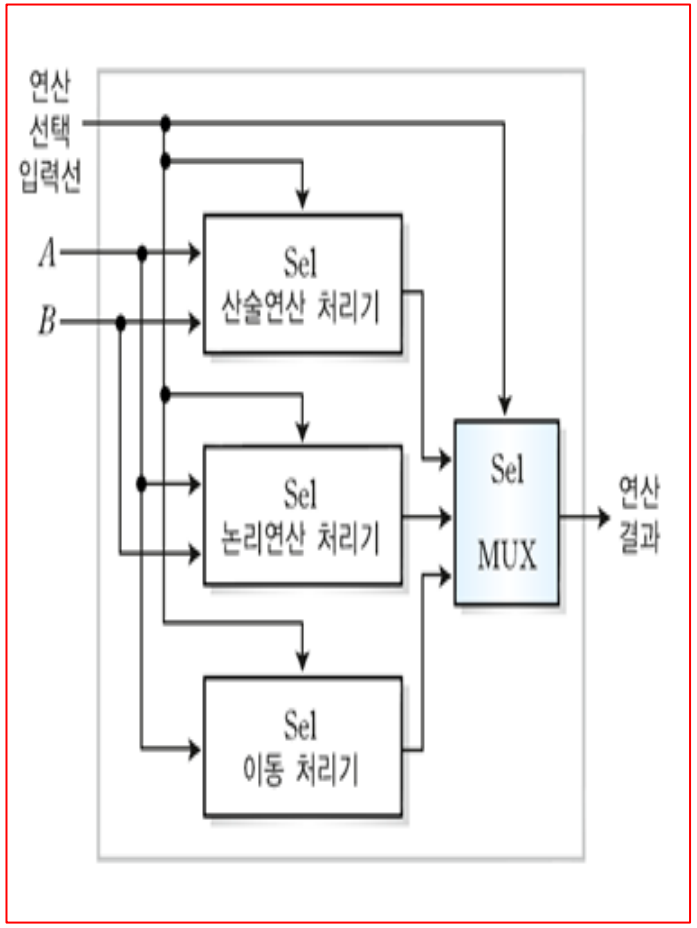
P152~p153

■ ALU (Arithmetic & Logic Unit)의 구성 요소

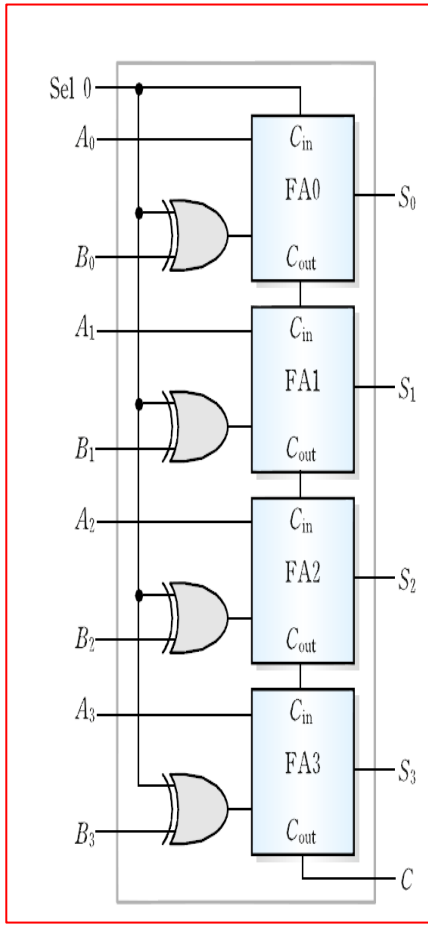
1) 산술(Arithmetic) 연산장치
   - ➢ 산술 연산들(+, -, ×, ÷)을 수행
2) 논리(Logic) 연산장치
   - ➢ 논리 연산들(AND, OR, NOT, XOR 등)을 수행
3) 시프트 레지스터(shift register)
   - ➢ 비트들을 좌측 혹은 우측으로 이동시키는 기능을 가진 레지스터
   - ➢ 산술연산의 시프트와 논리연산의 시프트는 다르게 동작
4) 보수기(complementer)
   - ➢ 2진 수치 데이터를 2의 보수로 변환(음수 ↔ 양수)
5) 상태 레지스터(status register)
   - ➢ 연산 결과의 결과 값 이외에 추가적으로 연산 결과의 상태를 나타내는 플래그(flag)들을 저장하는 레지스터
   - ➢ 이 레지스터는 ALU 내부에 있지 않고 단지 ALU가 이를 설정함



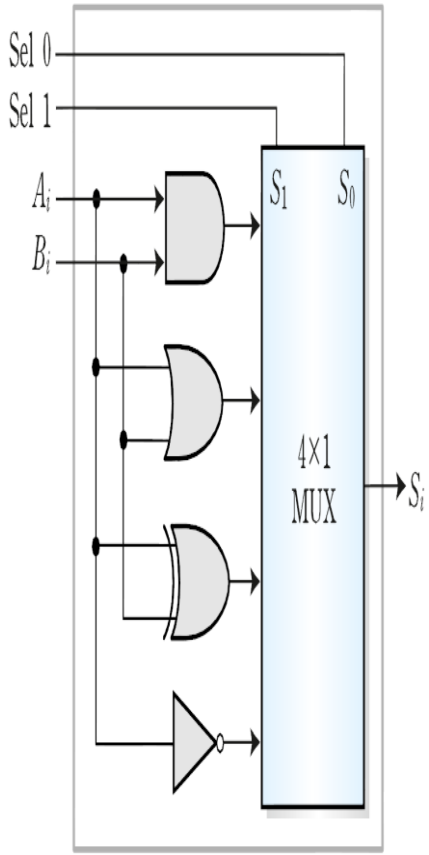
■ 4비트 ALU (Arithmetic & Logic Unit) 구성 예



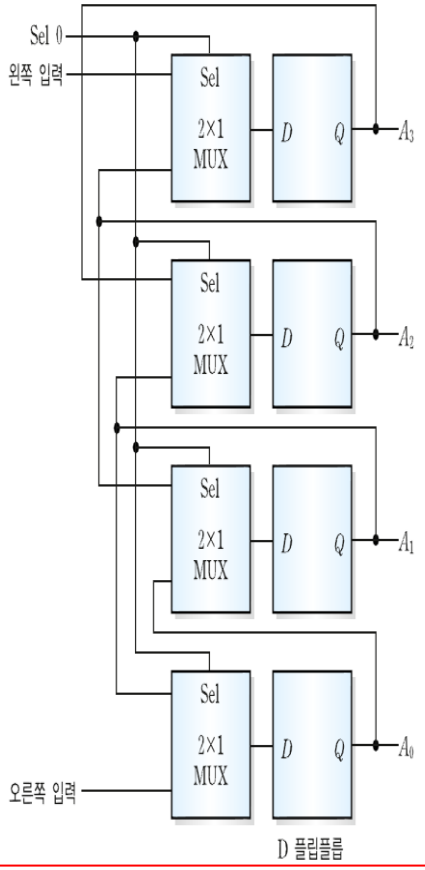
< 산술논리연산 장치 >



< 산술연산 장치 >



< 논리연산장치 >



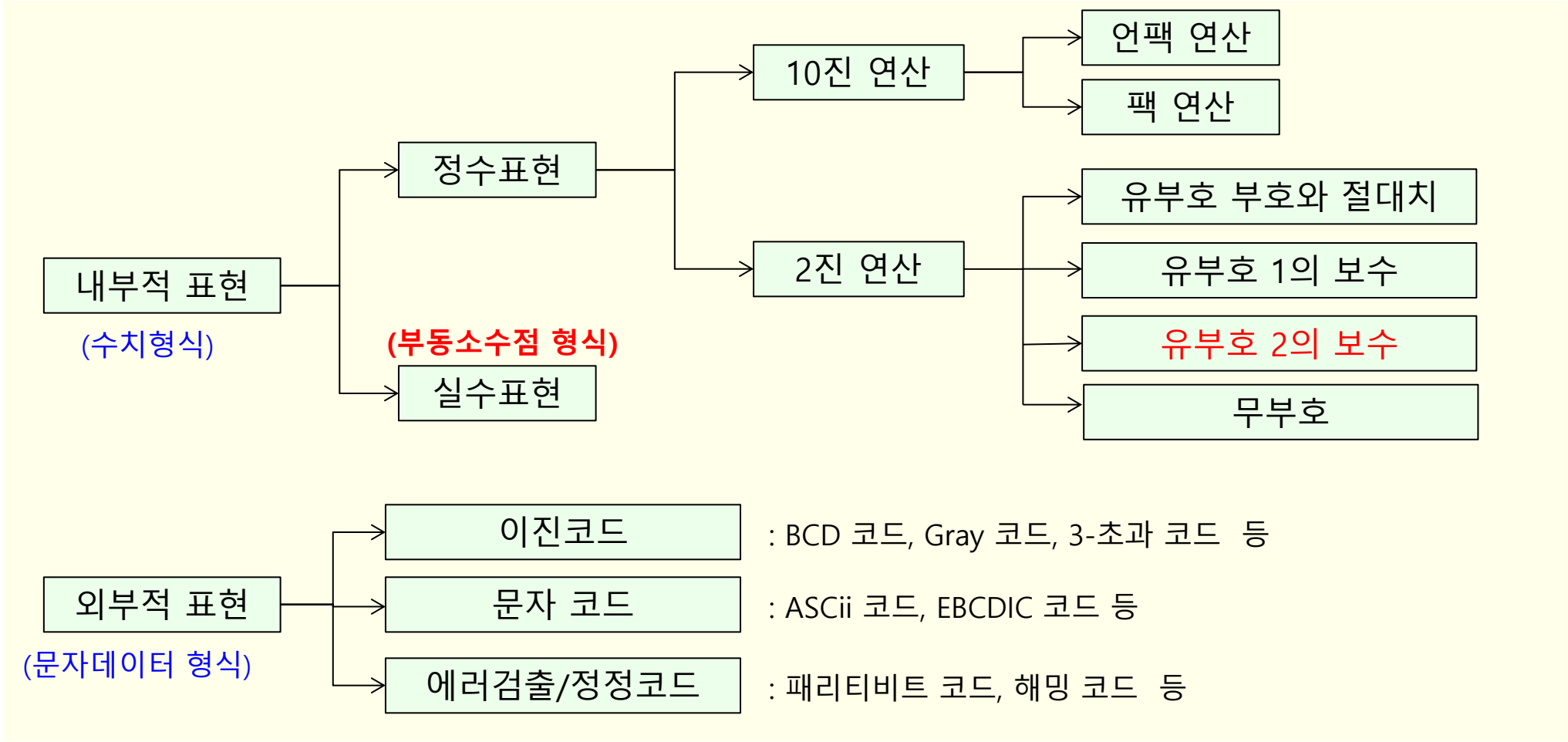
< 시프트 레지스터 >

■ ALU 연산: 자료의 성질에 따른 분류

✓ Shift와 Rotate를 산술연산과 논리 연산에 포함시킨 분류

구분	의미	종류
산술 연산 (수치적 연산)	정수, 부동소수점 방식의 수치 자료의 처리	- 사칙연산(ADD/SUB/MUL/DIV) - 산술적 Shift
논리 연산 (비수치적 연산)	비수치적 자료 처리	- 논리적 NOT, AND, OR, XOR - 보수(Complement) - 논리적 Shift - Rotate - 이동(Move)

■ 컴퓨터에서 사용되는 자료의 숫자/코드 체계



3.1 ALU

■ 컴퓨터에 사용되는 진법(=진수)

주의: 10진법 이외에는 반드시 진법(진수) 표시를 해야함!

10진법	2진법	16진법	8진법
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	8	10
9	1001	9	11
10	1010	A	12
11	1011	B	13
12	1100	C	14
13	1101	D	15
14	1110	E	16
15	1111	F	17

- ✓ 2진법 : Binary notation
  - ➢ 00000111_2 , $00000111_{(2)}$ 00000111b
- ✓ 10진법 : Decimal notation
  - ➢ 07_{10}, $07_{(10)}$, 07d, 07
- ✓ 8진법 : Octal notation
  - ➢ 07_8, $07_{(8)}$, 07o
- ✓ 16진법 : Hexadecimal notation
  - ➢ 07_{16}, $07_{(16)}$, 07h

2진법(진수): 컴퓨터가 사용하는 수 체계
10진법(진수): 사람이 사용하는 수 체계
8/16진법(진수):
- 컴퓨터와 인터페이스 할 경우 인간의 관점에서 사용
- Assembly 언어 사용 시, Memory 값 표현 시 등
- 2진수에 비해 자리수가 짧아 표현이 상대적으로 간결
- 8진수는 거의 사용하지 않음

3.2 정수의 표현

P154~p161

■ 2진수 체계

✓ **컴퓨터 등 디지털 시스템에서 사용하는 수 체계**

✓ 0, 1, 부호 및 소수점으로 수를 표현

➢ 예: 십진수 -13.62510 → 이진수 -1101.101_2

✓ 다음과 같은 3가지 표현 방법이 있음

1) 무부호 정수(Unsigned Integer)

❖ 부호가 없는 이진수로 구성되는 정수의 표현 방법 → 양의 정수만 존재

2) 유부호 정수(Signed Integer)

❖ 부호가 있는 이진수로 구성되는 정수의 표현 방법 → 양의 정수는 물론 음의 정수도 존재

❖ 3가지 표시 방법

☞ 부호와 절대치 표시방법, 1의 보수 표시방법, 2의 보수 표시방법

3) 부동 소수점(Floating-Point)

❖ 소수점 부분이 있거나,

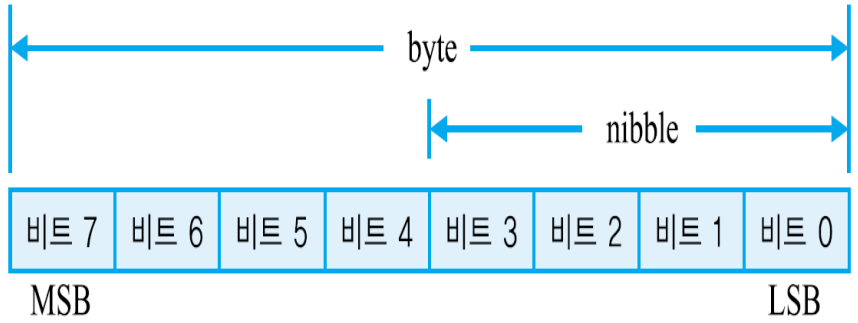
❖ 정수로 자리수가 초과되어 표현이 불가능한 매우 큰 수의 표현 방법

❖ IEEE754 단정도 표준, 배정도 표준 외

3.2 정수의 표현

■ 2진수에서 비트(bit)수와 수의 표현 범위

- ✓ 컴퓨터에서 사용되는 기본 수체계는 2진수
- ✓ Bit는 Binary Digit의 약자로 2진수를 표현하는 자리의 갯수(Digit)를 의미
  - ➢ 비트(자리) = bits =binary+digits
  - ➢ XXXb ← 3비트 (3자리의 이진수)
- ✓ Nibble(니블): 4 bit의 묶음
- ✓ Byte(바이트)
  - ➢ 8개의 bit들의 묶음 = 2개의 nibble
  - ➢ **MSB** (Most significant bit) → 2진 숫자에서 가장 높은 자리 비트
  - ➢ **LSB** (Least significant bit) → 2진 숫자에서 가장 낮은 자리 비트
- ✓ Word(워드)
  - ➢ **CPU가 한번에 처리할 수 있는 자리 수** (CPU 내부 구조와 동일)
  - ➢ 16bit CPU의 1 워드 = 2byte(16bit), 32bit CPU의 1워드 = 4byte(32bit), 64bit CPU의 1워드 = 8byte(64bit)



3.2 정수의 표현

■ 2진수에서 비트(bit)수와 수의 표현 범위 - 계속

✓ 2^n가지의 정보를 표시 → **n** bit 필요

✓ n bit 양의 정수의 범위 : **0 ~ 2^n-1**

디지털 수 체계에서는 0부터 시작되는 것에 주의

➢ Bit수가 많아질 수록 표현범위가 넓어짐

➢ 예) Bit 수 표시가능 수의 개수(=2^n) 표현 범위

❖ 1	2 (2^1)	0 ~ 1 (2^1-1)
❖ 2	4 (2^2)	0 ~ 3 (2^2-1)
❖ 3	8 (2^3)	0 ~ 7 (2^3-1)
❖ 4	16 (2^4)	0 ~ 15 (2^4-1)
❖ 8	256 (2^8)	0 ~ 255 (2^8-1)
❖ 16	65536 (2^{16})	0 ~ 65535 (2^{16}-1)
❖ 32	4294967296 (2^{32})	0 ~ 4294967295 (2^{32}-1)

3.2 정수의 표현

■ 2진수에서 비트(bit)수와 수의 표현 범위 - 계속

- ✓ 2^n가지의 정보를 표시 → n bit 필요
- ✓ 컴퓨터에서 주기억장치의 주소 지정 예 (1주소에 1byte 지정시)
  - ➢ 16bit 주소선 : 2^{16} 개의 저장장소 → 64KByte = 64 x 1024 = 64Kbyte
  - ➢ 32bit 주소선 : 2^{32} 개의 저장장소 → 4GByte = 4 x 1024 x 1024 x 1024 = 4Gbyte
- ✓ 주요 단위
  - ➢ 2진수는 **IEC 단위계**를 사용하며 10진수의 SI 단위계와는 값이 다름
    - ❖ 2^{10} = 1024 = 1kibi(키비) = 1Ki
    - ❖ 2^{20} = 1024K = 정식 명칭은 1mebi(메비) = 1Mi
    - ❖ 2^{30} = 1024M = 1gibi(키비) = 1Gi
    - ❖ 2^{40} = 1024G = 1tebi(테비) = 1Ti
  - ➢ **bi는 binary의 약자인데 일반적으로 i를 빼고 사용하는 경향이 있음**
    - ❖ 1Mib=1Mebibit(메비비트)를 그냥 통상적으로 1Mb(메가비트)로 부름
    - ❖ 1MiB=1Mebibit(메비바이트)를 그냥 통상적으로 1MB(메가바이트)로 부름

소문자 b는 bit, 대문자 B는 Byte를 의미

SI(10진 단위)			IEC(2진 단위)			
값	기호	이름	값	기호	이름	10진 변환 크기
$(10^3)^1=10^3$	k, K	kilo-	$(2^{10})^1=2^{10}\simeq10^{3.01}$	Ki	kibi-	1,024
$(10^3)^2=10^6$	M	mega-	$(2^{10})^2=2^{20}\simeq10^{6.02}$	Mi	mebi-	1,048,576
$(10^3)^3=10^9$	G	giga-	$(2^{10})^3=2^{30}\simeq10^{9.03}$	Gi	gibi-	1,073,741,824
$(10^3)^4=10^{12}$	T	tera-	$(2^{10})^4=2^{40}\simeq10^{12.04}$	Ti	tebi-	1,099,511,627,776
$(10^3)^5=10^{15}$	P	peta-	$(2^{10})^5=2^{50}\simeq10^{15.05}$	Pi	pebi-	1,125,899,906,842,624
$(10^3)^6=10^{18}$	E	exa-	$(2^{10})^6=2^{60}\simeq10^{18.06}$	Ei	exbi-	1,152,921,504,606,846,976
$(10^3)^7=10^{21}$	Z	zetta-	$(2^{10})^7=2^{70}\simeq10^{21.07}$	Zi	zebi-	1,180,591,620,717,411,303,424
$(10^3)^8=10^{24}$	Y	yotta-	$(2^{10})^8=2^{80}\simeq10^{24.08}$	Yi	yobi-	1,208,925,819,614,629,174,706,176

HDD 용량 1TB = 10^{12} = 1,000,000,000,000 byte

RAM 1TB = 2^{40} = 1,099,511,627,776 byte

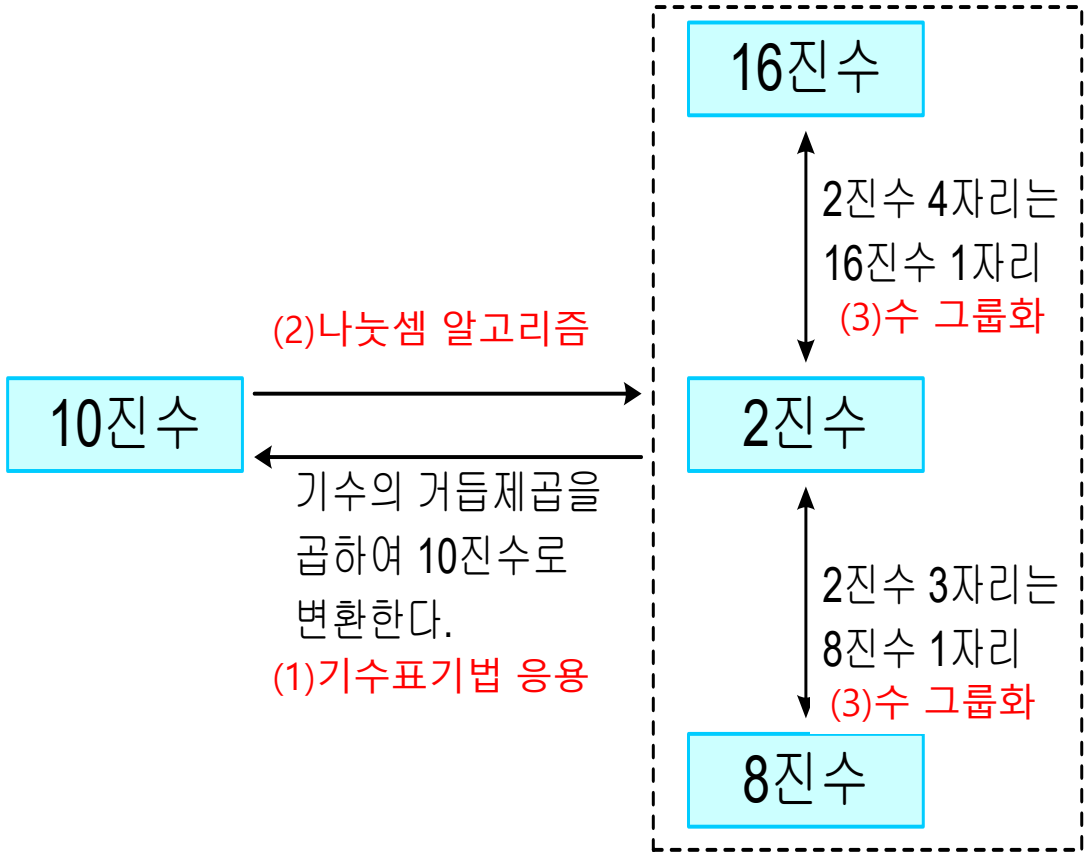
3.2 정수의 표현

■ 기수(진수) 변환 요약

교재에는 없으나 매우 중요한 내용이며 정수는 물론 소수점이 있는 수체계에서도 사용

✓ 2진수-8진수-16진수-10진수 상호 변환

10진수	2진수	8진수	16진수
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F



3.2 정수의 표현

P154~p161

1) 위치 표기법 (기수 표기법)

위치(기수) 표기법 → 임의의 진수에서 10진수로의 변환에 많이 사용

$(N)_r$ = 정수부 + 소수부

$= a_n \times r^n + a_{n-1} \times r^{n-1} + \cdots + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + a_{-2} \times r^{-2} + \cdots + a_{-m} \times r^{-m}$

- ✓ **a** : 계수(Coefficients) (0<= a < r 인 정수)
- ✓ **n,m** : 지수 / 거듭제곱수 / 멱승
- ✓ **r** : 기수(基數)/ 밑수 / 진수/ Base / Radix $r^0 = 1$
  - ➢ (r = 10 → 10진수, r = 16 → 16진수, r = 8 → 8진수, r = 2 → 2진수)

- ✓ $654.236_{10} = 6 \times 10^2 + 5 \times 10^1 + 4 \times 10^0 + 2 \times 10^{-1} + 3 \times 10^{-2} + 6 \times 10^{-3}$
- ✓ $7642_{10} = 7 \times 10^3 + 6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$
- ✓ 2진수 $101111_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \ (= 32 + 8 + 4 + 2 + 1 = 47_{10})$

10진수의 경우는 기수를 **10**으로 표현

2진수의 경우는 기수를 **2**로 표현

1) 위치 표기법 (기수 표기법) - 계속

✓ 진법을 나타내는 기본수를 기수(基數, radix)라 한다.

✓ 10이 기수인 수를 10진법, 2가 기수인 수를 2진법, 12가 기수인 수를 12진법이라 한다 (진법 ≒ 진수)

1) 10진수 표현법

➢ 10진수 : **기수가 10인 수**

➢ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9의 **10개 수로 표현**

$$9345.35 = 9\times1000 + 3\times100 + 4\times10 + 5\times1 + 3\times0.1 + 5\times0.01$$
$$= 9\times10^{3} + 3\times10^{2} + 4\times10^{1} + 5\times10^{0} + 3\times10^{-1} + 5\times10^{-2}$$

➢ 바빌로니아인 : 60진법을 사용(기원전 4000~3000년)

➢ 고대 로마의 기수법에는 5진법을 사용

➢ 10진법의 아라비아 숫자는 인도에서 기원전 2세기에 발명

3.2 정수의 표현

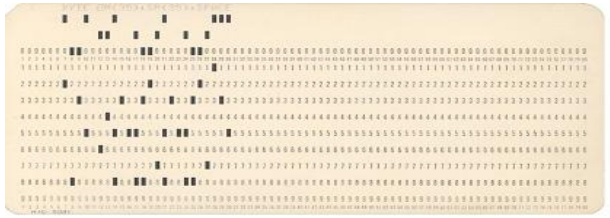
1) 위치 표기법 (기수 표기법) - 계속

2) 2진수 표현법

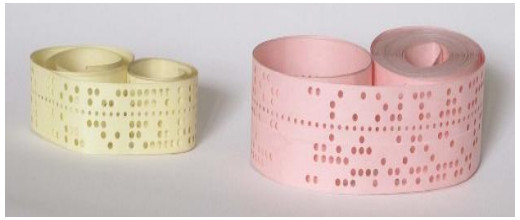
- 2진수 : **기수가 2인 수**
- 0, 1의 2개 수로 표현
- 숫자 뒤에 b(binary) 또는 아래첨자로 $_2$ 또는 $_{(2)}$를 붙여 2진법 표시

$$
\begin{aligned}
1010.1011_{(2)} &= 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 0\times 2^0 + 1\times \frac{1}{2^1} + 0\times \frac{1}{2^2} + 1\times \frac{1}{2^3} + 1\times \frac{1}{2^4} \\
&= 1\times 2^3 + 0\times 2^2 + 1\times 2^1 + 0\times 2^0 + 1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3} + 1\times 2^{-4} \\
&= 8 + 0 + 2 + 0 + 0.5 + 0 + 0.125 + 0.0625 \\
&= 10.6875
\end{aligned}
$$

기수 표기법
(x 승의 합으로 표시)



Punch card



종이 테이프

1) 위치 표기법 (기수 표기법) - 계속

3) 8진수 표현법

- 0,1,2,3,4,5,6,7까지 여덟 가지의 수로 표현, 기수가 8인 수
- 숫자 뒤에 o(octal) 또는 아래첨자로 $_8$ 또는 $_{(8)}$를 붙여 8진법 표시

$$607.36_{(8)} = 6\times100_{(8)} + 0\times10_{(8)} + 7\times1_{(8)} + 3\times0.1_{(8)} + 6\times0.01_{(8)}$$
$$= 6\times8^2 + 0\times8^1 + 7\times8^0 + 3\times8^{-1} + 6\times8^{-2}$$

기수 표기법

- 2진수 3자리는 8진수 1자리 : $2^3 = 8^1$ 수 그룹화

$$10101110100010.0111111_{(2)} = 10\ 101\ 110\ 100\ 010.011\ 111\ 1_{(2)}$$
$$= 010\ 101\ 110\ 100\ 010.011\ 111\ 100_{(2)}$$
$$= 2\ 5\ 6\ 4\ 2.\ 3\ 7\ 4_{(8)}$$

- 8진수에 해당하는 2진수

8진수	0	1	2	3	4	5	6	7
2진수	000	001	010	011	100	101	110	111

1) 위치 표기법 (기수 표기법) - 계속

4) 16진수 표현법

- 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F까지 16 가지의 수와 기호로 표현, 기수는 16
- 숫자 뒤에 h(hexadecimal) 또는 아래첨자로 $_{16}$ 또는 $_{(16)}$를 붙임

$$\begin{aligned} 6C7.3A_{(16)} &= 6\times16^2 + C\times16^1 + 7\times16^0 + 3\times16^{-1} + A\times16^{-2} \\ &= 6\times256+12\times16+7\times1+3\times0.0625+10\times0.0039625 \\ &= 1536+192+7+0.1875+0.0390625 \\ &= 1735.2265625 \end{aligned}$$

기수 표기법

- 2진수 4자리는 16진수 1자리 : $2^4 = 16^1$

수 그룹화

$$\begin{aligned} 101011110100010.0111111_{(2)} &= \quad 10 \quad 1011 \quad 1010 \quad 0010.0111 \quad 111_{(2)} \\ &= 0010 \quad 1011 \quad 1010 \quad 0010.0111 \quad 1110_{(2)} \\ &= \quad 2 \quad B \quad A \quad 2 . \quad 7 \quad E_{(16)} \end{aligned}$$

3.2 정수의 표현

2) 나눗셈 알고리즘

10진수 → N 진수 계산에 적용

- ✓ 예) $41.6875_{10} = 101001.1011_{2}$, $153.513_{10} = 231.406517_{8}$

- ✓ 정수부분과 소수부분의 계산이 다르다
- ✓ 정수부분(N)
  - ➢ 몫이 0이 될 때까지 N(기수, 2진법의 경우 2)로 **나누어 나머지**를 모은다
- ✓ 소수부분(N)
  - ➢ 소수점 이하가 0이 될 때까지 N(기수, 2진법의 경우 2)으로 **곱해 정수부분**만을 모은다
  - ➢ 10진수의 소수부분은 정확한 N 진수로 변환이 안되는 경우가 많음
    - ❖ 10진수 75.6을 2진수로 변환하는 경우 → 소수부분의 자리수가 계속되는 문제됨
    - ❖ 따라서 소수 몇 번째 자리까지 구하라고 명시됨

2) 나눗셈 알고리즘 예

✓ 10진수 → 2진수 변환 나눗셈 알고리즘의 예제

- 정수부분과 소수부분으로 나누어 변환
- 정수부분은 2로 나누고, 소수부분은 2를 곱한다.
- 10진수 75.6875를 2진수로 변환

2	75	나머지 ⟶	2진수
2	37	··· 1 ⟶	1
2	18	··· 1 ⟶	11
2	9	··· 0 ⟶	011
2	4	··· 1 ⟶	1011
2	2	··· 0 ⟶	01011
2	1	··· 0 ⟶	001011
	0	··· 1 ⟶	1001011
	몫		

2진수 ⟵	정수	소수	
	0.	6875	
		X 2	
0.1 ⟵	1.	3750	곱셈결과 정수를 적는다.
		X 2	
0.10 ⟵	0.	7500	
		X 2	
0.101 ⟵	1.	5000	
		X 2	소수부분이 0이 될 때까지 계산한다.
0.1011 ⟵	1.	0	

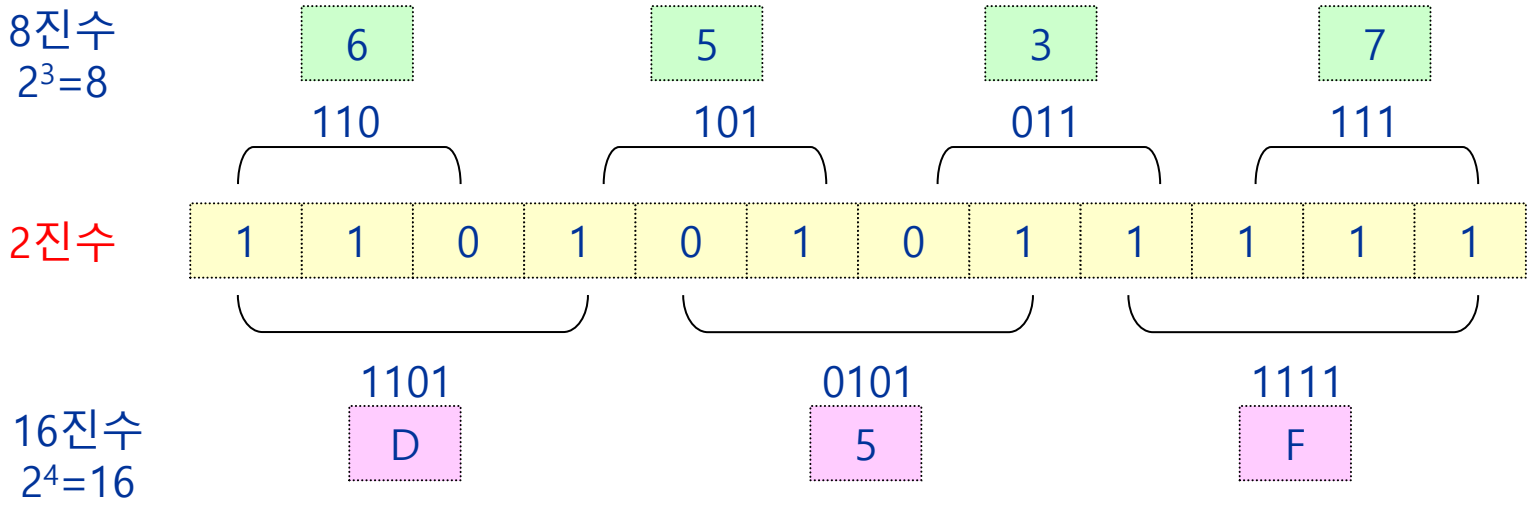
$$75.6875_{(10)} = 1001011.1011_{(2)}$$

3) 수 그룹화에 의한 기수변환

- ✓ 2진수 3자리는 8진수, 2진수 4자리는 16진수인 원리를 이용 (**수 그룹화**)
- ✓ 2진수로 변환 후 소수점을 기준으로 아래(소수점 부분)와 위(정수 부분) 방향으로 해당 자릿수로 묶어(수 그룹화) 처리
  - ➢ 8진수 ↔ 2진수 3자리, 16진수 ↔ 2진수 4자리

2진수 ↔ 8진수, 2진수 ↔ 16진수, 8진수 ↔ 16진수 계산에 적용

- ✓ 예제 : 16진수에서 8진수로 변환
  - ➢ $D5F_{16}$ = $\underline{1101}$ $\underline{0101}$ $\underline{1111}_2$ = 110101011111_2 = $\underline{110}$ $\underline{101}$ $\underline{011}$ $\underline{111}_2$ = 6537_8



3) 수 그룹화에 의한 기수변환

✓ **상호변환 예 (10진→2진→8진)**

$$75.6875 = 1001011.1011_{(2)}$$
$$= 001\ \ 001\ \ 011.101\ \ 100_{(2)}$$
$$=\ \ 1\ \ \ \ 1\ \ \ \ 3.\ \ \ 5\ \ \ \ 4_{(8)}$$

10진→2진(나눗셈 알고리즘)→8진(수그룹화)
3자리씩 나눔

✓ **상호변환 예 (10진→2진→16진)**

$$75.6875 = 1001011.1011_{(2)}$$
$$= 0100\ \ 1011.1011_{(2)}$$
$$=\ \ \ \ 4\ \ \ \ B.\ \ \ \ B_{(16)}$$

10진 → 2진(나눗셈 알고리즘) → 16진(수 그룹화)
4자리씩 나눔

✓ **상호변환 예 (수 그룹화)**

$$367.75_{(8)} = 011\ \ 110\ \ 111.111\ \ 101_{(2)}$$

8진수 1자리 = 2진수 3자리

$$9A3.50F3_{(16)} = 1001\ \ 1010\ \ 0011.0101\ \ 0000\ \ 1111\ \ 0011_{(2)}$$

16진수 1자리 = 2진수 4자리

3.2 정수의 표현

P154~p161

■ 음수의 표현 (유부호)

유부호 (정수) 수체계에서 MSB는 항상 부호 비트임 (0 = 양수, 1 = 음수)

1) 부호화-크기 표현(signed-magnitude representation)

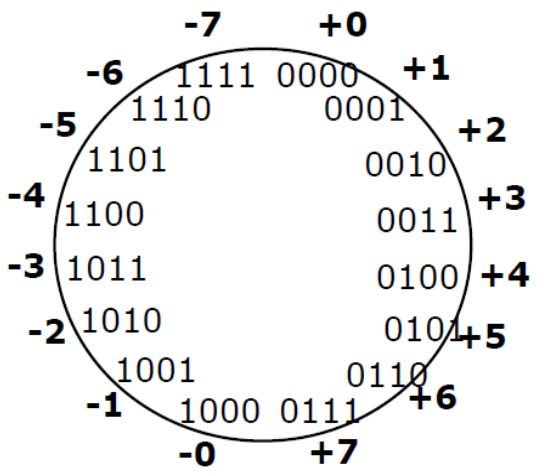
- 최상위 비트를 제외한 나머지 비트가 크기를 나타냄

2) 1의 보수 표현(1's complement representation)

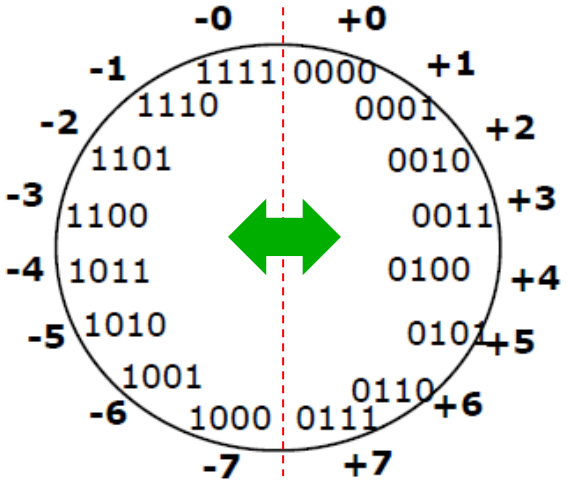
- 양수, 음수의 변환은 모든 0 → 1, 1 → 0으로 바꾼 것이 크기를 나타냄

3) 2의 보수 표현(2's complement representation)

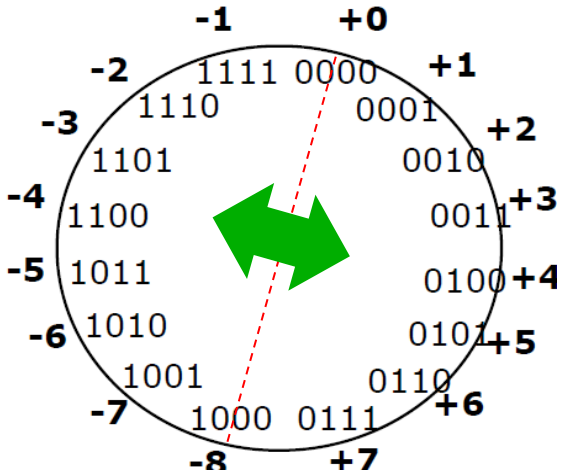
- 양수, 음수의 변환은 모든 0 → 1, 1 → 0으로 바꾼 후 1을 더한 값



4비트 부호화-크기 표현



4비트 1의 보수 표현



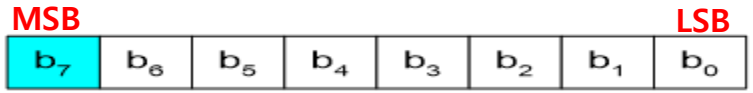
4비트 2의 보수 표현

3.2.1 부호화-크기 표현

P154~p161

■ 부호화-크기 표현

✓ 맨 좌측 비트(MSB)는 부호 비트, 나머지 (n-1)개의 비트들은 수의 크기(magnitude)를 나타내는 표현 방식



- +9 = 00001001b -9 = 10001001b
- +35 = 00100011b -35 = 10100011b

✓ 부호화-크기로 표현된 2진수(a_{n-1} a_{n-2} … a_1 a_0)를 10진수로 변환

- $A = (-1)^{an-1} (a_{n-2} \times 2^{n-2} + a_{n-3} \times 2^{n-3} + \ldots + a_1 \times 2^1 + a_0 \times 2^0)$
- 예: 0 0100011b = $(-1)^0(0\times2^6 + 1\times2^5 + 0\times2^4 + 0\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0) = (32 + 2 + 1) = 35$
- 예: 1 0001001b = $(-1)^1(0\times2^6 + 0\times2^5 + 0\times2^4 + 1\times2^3 + 0\times2^2 + 0\times2^1 + 1\times2^0) = -(8 + 1) = -9$

✓ 결점

- 덧셈과 뺄셈을 수행하기 위해서는 부호 비트와 크기를 비교하여 처리하는 복잡한 과정 필요
- 0에 대한 표현이 두 개 존재
  - 00000000b = +0, 10000000b = -0
  - n-비트 단어로 표현할 수 있는 수들이 2^n 개가 아닌, $(2^n - 1)$개로 감소

3.2.1 부호화-크기 표현

P154~p161

■ 부호화-크기 표현(4비트)

✓ **반대 부호의 값 얻는 방법**

	부호 비트	절대치				
0	0	0	0	0		+0
	0	0	0	1		+1
	0	0	1	0		+2
	0	0	1	1		+3
+	0	1	0	0		+4
	0	1	0	1		+5
	0	1	1	0		+6
	0	1	1	1		+7
0	1	0	0	0		-0
	1	0	0	1		-1
	1	0	1	0		-2
	1	0	1	1		-3
-	1	1	0	0		-4
	1	1	0	1		-5
	1	1	1	0		-6
	1	1	1	1		-7

절대치는 같고 부호만 다름

3.2.2 보수 표현

■ 보수(補數, complement)

- ✓ 사칙연산을 하려면 정수 범위라도 음수를 표현할 수 있어야 하는데 컴퓨터는 2진수의 음의 정수 표기 방법에서 보수를 사용
- ✓ 보수는 '**보충하는 수**' 라는 의미이며 '**n의 보수**'와 '**n-1의 보수**' 의 2가지 종류가 있다
  - ➢ 2진수라면 2(n)의 보수와 1(n-1)의 보수가 존재하며 10진수라면 10(n)의 보수, 9(n-1)의 보수가 존재
- ✓ **n-1의 보수**
  - ➢ **자릿수를 유지하며 해당 수를 뒤집은 수**(다르게 표현하면 더했을 때 자릿수가 올라가게 하는 수에서 1을 뺀 값)
  - ➢ 10진수(9의 보수)의 경우 한 자릿수 1을 뒤집으면 8, 2를 뒤집으면 7, 두 자릿수 99의 9의 보수는 0, 45는 54, 38은 61
  - ➢ 2진수(1의 보수)의 경우는 0을 뒤집으면 1, 1을 뒤집으면 0이기 때문에 1101b의 1의 보수는 0010b, 11111b의 보수는 00000b
- ✓ **n의 보수**
  - ➢ **바로 더했을 때 자릿수가 올라가게 하는 수**(다르게 표현하면 n-1의 보수에서 1을 더한 값)
  - ➢ 10진수의 경우 45에 더했을 때 100이 되게 하는 수를 찾으면 곧 10의 보수가 되며 55가 45의 10의 보수
  - ➢ 2진수의 경우 1101b의 2의 보수를 구하면 0011b → **1**0000b (기존 자리의 모든 값이 0이며 자릿수가 하나 늘어 났음)
    - ❖ 이 경우 쉽게 구하는 방법은 원래의 수에서 0과 1을 모두 바꾸고(1의 보수) 여기에 추가로 1을 더하면 됨
    - ❖ 1101b → 0010b → 0010b + 1b → 0011b

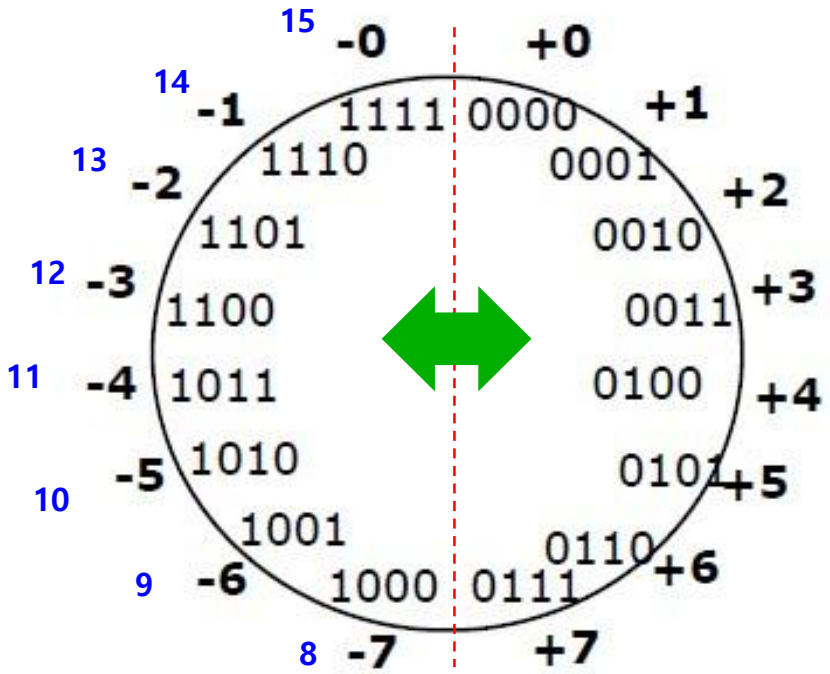
3.2.2 보수 표현

P154~p161

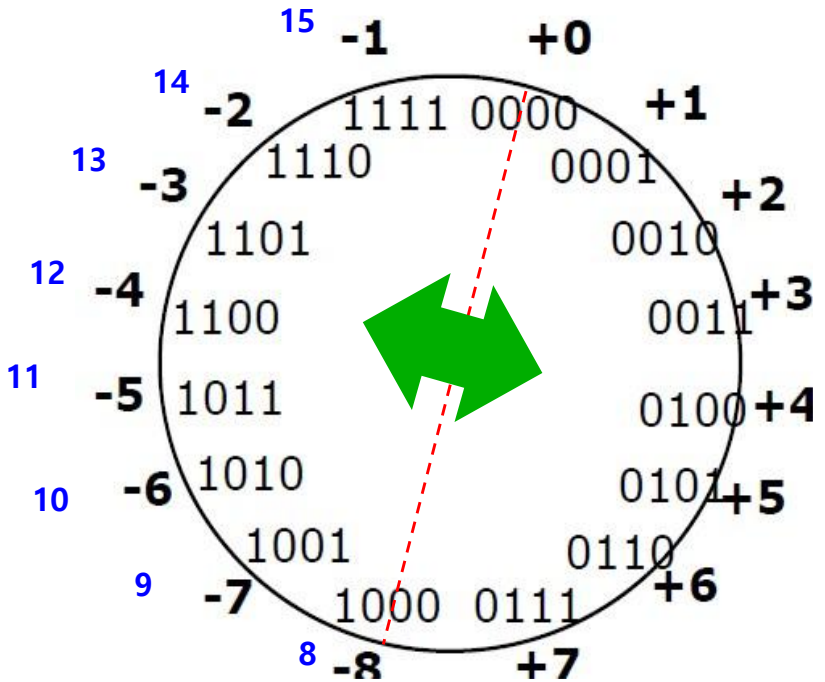
■ 컴퓨터에서 보수의 활용

- ✓ 컴퓨터에서는 보수를 이용해 음의 정수를 표현한다
- ✓ **아래 그림처럼 수 체계를 정의해서 사용** → 4자리 2진수의 경우임
  - ➢ 양수를 보수로 바꾸면 음수, 음수를 보수로 바꾸면 양수 양수 ⟷ (보수) 음수

우리는 이진수에 대한 학습만 한다. → 즉 1(n-1)의 보수 및 2(n)의 보수
보수 체계에서는 자릿수가 매우 중요하며, 자릿수를 지정해 주지 않으면 보수 계산시에 엉뚱한 결과가 나오기 때문에 문제에서는 항상 자릿수를 지정해 줌



4비트 2진수(십진수 16) 시스템에서 1(n-1)의 보수



4비트 2진수(십진수 16) 시스템에서 2(n)의 보수

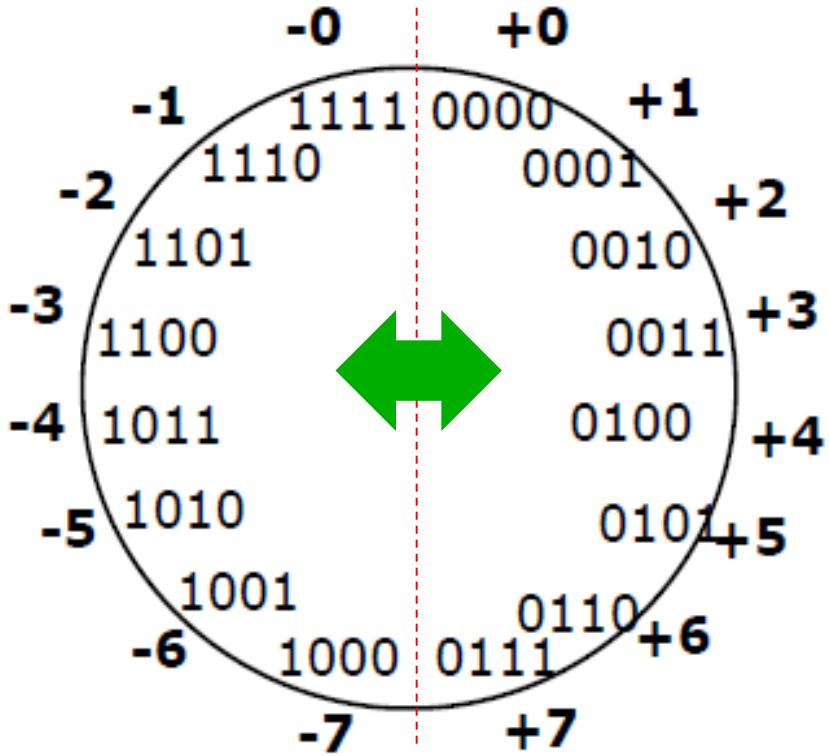
3.2.2 보수 표현

P154~p161

■ 1의 보수(1's complement) 표현

잘 사용하지 않고 주로 2의 보수를 사용

- ✓ 양수 : 부호화 크기법과 동일
- ✓ 음수 : 양수의 1의 보수로 표현
- ✓ 1의 보수에서 양수/음수 변환법
  - ➢ 직관적 방법 : **0을 1로, 1을 0으로 바꾼다**
    - ❖ 예) 0111(7) → 1000(-7)
  - ➢ 수식적 방법 : $N' = (2^n - 1) - N$ (n=자릿수)
- ✓ 단점 : **2개의 0 표현**
- ✓ 가능 숫자 범위
  - ➢ **$-(2^{n-1}-1) \sim +(2^{n-1}-1)$**
  - ➢ 4진수: $-(2^{4-1}-1) \sim +(2^{4-1}-1) = -7 \sim +7$
  - ➢ 8진수: $-(2^{8-1}-1) \sim +(2^{8-1}-1) = -127 \sim +127$



3.2.2 보수 표현

P154~p161

■ 1의 보수(1's complement) 표현 (4비트)

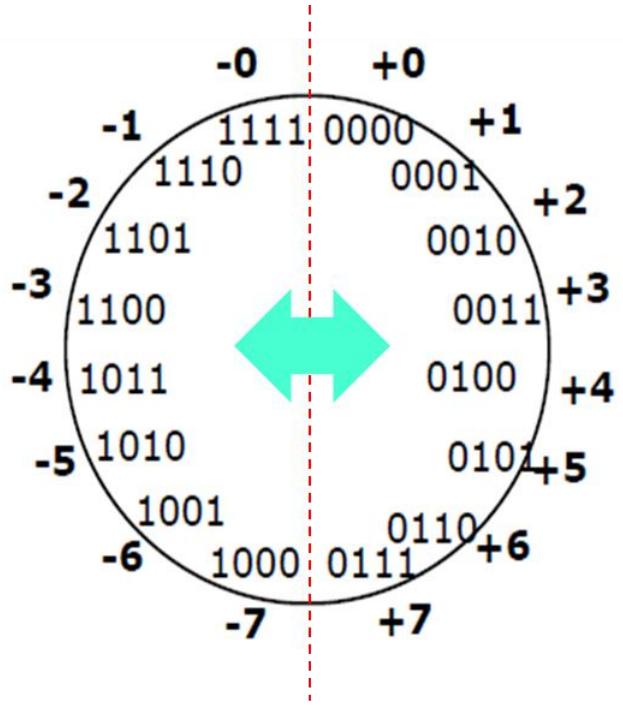
✓ 반대 부호의 값 얻는 방법

	부호 비트					
0	0	0	0	0		+0
	0	0	0	1		+1
	0	0	1	0		+2
	0	0	1	1		+3
+	0	1	0	0		+4
	0	1	0	1		+5
	0	1	1	0		+6
	0	1	1	1		+7
	1	0	0	0		-7
	1	0	0	1		-6
	1	0	1	0		-5
-	1	0	1	1		-4
	1	1	0	0		-3
	1	1	0	1		-2
	1	1	1	0		-1
0	1	1	1	1		-0

0과 1을 서로 바꿈

1의 보수

	부호 비트					
	1	1	1	1		-0
	1	1	1	0		-1
	1	1	0	1		-2
	1	1	0	0		-3
	1	0	1	1		-4
	1	0	1	0		-5
	1	0	0	1		-6
	1	0	0	0		-7
	0	1	1	1		+7
	0	1	1	0		+6
	0	1	0	1		+5
	0	1	0	0		+4
	0	0	1	1		+3
	0	0	1	0		+2
	0	0	0	1		+1
	0	0	0	0		+0

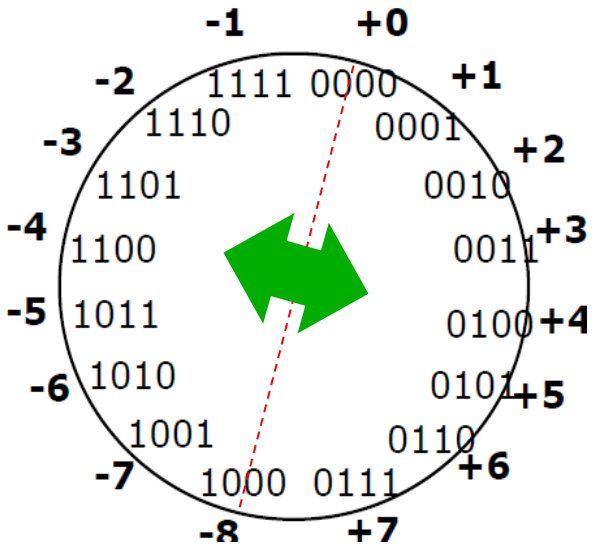


3.2.2 보수 표현

P154~p161

■ 2의 보수(2's complement) 표현

매우 중요



- ✓ 양수 : 부호화 크기법과 동일
- ✓ 음수 : 양수의 2의 보수로 표현
- ✓ 2의 보수에서 양수/음수 변환법
  - ➢ **0을 1로, 1을 0으로 바꾼 후에 1을 더한다**
  - ➢ 예) 0111(7) → 1000+1 → 1001(-7)
- ✓ 장점 : **숫자를 하나 더 표현**
- ✓ 가능 숫자 범위 : **$-(2^{n-1}) \sim +(2^{n-1}-1)$**
  - ➢ 4진수: $-(2^{4-1}) \sim +(2^{4-1}-1) = -8 \sim +7$
  - ➢ 8진수: $-(2^{8-1}) \sim +(2^{8-1}-1) = -128 \sim +127$
- ✓ **컴퓨터에서는 2의 보수를 사용한다. 왜일까?**
  - ➢ 부호화 크기법 보다 **연산이 쉽고** 1의 보수보다 표현할 수 있는 **수가 하나 더 많다**
  - ➢ **H/W가 간편**
    - ❖ 가산기(ALU) 앞에 NOT 회로와 +1 회로를 추가해 가산기로 감산을 실시

3.2.2 보수 표현

P154~p161

■ 2의 보수(2's complement) 표현 (4비트)

매우 중요

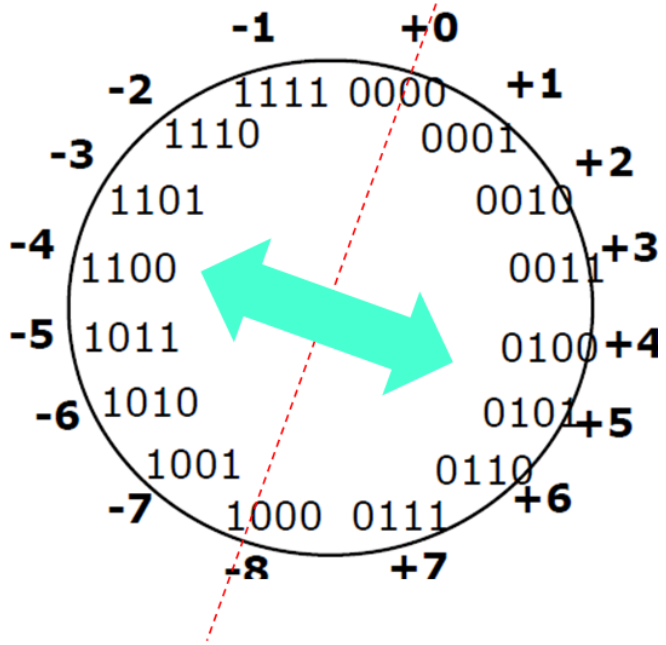
✓ 반대 부호의 값 얻는 방법

	부호 비트					
0	0	0	0	0		0
	0	0	0	1		+1
	0	0	1	0		+2
	0	0	1	1		+3
+	0	1	0	0		+4
	0	1	0	1		+5
	0	1	1	0		+6
	0	1	1	1		+7
	1	0	0	0		-8
	1	0	0	1		-7
	1	0	1	0		-6
	1	0	1	1		-5
-	1	1	0	0		-4
	1	1	0	1		-3
	1	1	1	0		-2
	1	1	1	1		-1

1의 보수로 바꾼 후 1을 더함

2의 보수

부호 비트					
0	0	0	0		0
1	1	1	1		-1
1	1	1	0		-2
1	1	0	1		-3
1	1	0	0		-4
1	0	1	1		-5
1	0	1	0		-6
1	0	0	1		-7
1	0	0	0		-8
0	1	1	1		+7
0	1	1	0		+6
0	1	0	1		+5
0	1	0	0		+4
0	0	1	1		+3
0	0	1	0		+2
0	0	0	1		+1



계산이 쉽고 수의 표현 범위가 하나 더 많아서 대부분의 컴퓨터에서 2의 보수를 사용

3.2.2 보수 표현

P154~p161

■ 정리: 부호화 2진수 표시에서 음수의 크기 구하는 방법

매우 중요

✓ 부호화 2진수에서 첫 bit(MSB)가 1 이면 음수를 표시하는데, 음수일 경우

1) 부호화 크기법 : 최상위 비트를 제외한 나머지 비트가 크기를 나타냄

2) 1의 보수 표기법 : 모든 0 → 1, 1 → 0으로 바꾼 것이 크기(양수)를 나타냄

3) 2의 보수 표기법 : 모든 0 → 1, 1 → 0으로 바꾼 후 1을 더한 값(양수)

✓ 예: 유부호 2진수 4자릿수 1101_2의 표기법에 따른 크기 구하기

자릿수 주의

- 부호화 크기 표현인 경우: 1101의 크기는 101=5 이므로 -5
- 1의 보수 표현인 경우: 1101의 크기는 0010 = 2 이므로 -2
- 2의 보수 표현인 경우: 1101의 크기는 0010 + 1 = 0011 =3 이므로 -3

✓ 예: 8비트 2진수의 표현 예 지금 풀어 봅시다

수	무부호	부호화 크기	2의 보수	1의 보수
36 (24h)	00100100	00100100	00100100	00100100
-36 (-24h))	-	10100100	11011100	11011011
20 (14h)	00010100	00010100	00010100	00010100
-20 (-14h)	-	10010100	11101100	11101011

3.2.2 보수 표현

P154~p161

■ 8-비트 2진수로 표현할 수 있는 10진수의 범위

- ✓ 1의 보수 : $-(2^7 - 1) \sim +(2^7 - 1)$
- ✓ 2의 보수 : $-2^7 \sim +(2^7 - 1)$

10진수	1의 보수	2의 보수
127	01111111	01111111
126	01111110	01111110
⋮	⋮	⋮
2	00000010	00000010
1	00000001	00000001
+0	00000000	00000000
−0	11111111	—
−1	11111110	11111111
−2	11111101	11111110
⋮	⋮	⋮
−126	10000001	10000010
−127	10000000	10000001
−128	—	10000000

3.2.2 보수 표현

P154~p161

■ 2의 보수 → 10진수 변환

✓ 2의 보수로 표현된 양수($a_{n-1} = 0$)를 10진수로 변환하는 방법

➢ $A = a_{n-2} \times 2^{n-2} + a_{n-3} \times 2^{n-3} + \dots a_1 \times 2^1 + a_0 \times 2^0$

✓ 2의 보수로 표현된 음수($a_{n-1} = 1$)를 10진수로 변환하는 방법

➢ $A = -2^{n-1} + (a_{n-2} \times 2^{n-2} + a_{n-3} \times 2^{n-3} + \dots a_1 \times 2^1 + a_0 \times 2^0)$

➢ 예

❖ $10101110b = -128 + (1 \times 2^5 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1)$

$= -128 + (32 + 8 + 4 + 2) = -82$

➢ 다른 방법

❖ 10101110b → 01010010b 으로 먼저 변환한 후, 음수 표시

❖ $01010010b = -(1 \times 2^6 + 1 \times 2^4 + 1 \times 2^1)$

$= -(64 + 16 + 2) = -82$

3.2.3 부호-비트 확장 (Bit Extension)

P154~p161

■ 데이터의 길이(비트 수)를 늘리는 방법

✓ 필요성

➢ 데이터를 더 많은 비트의 레지스터에 저장하거나, 더 긴 데이터와 연산을 수행하기 위해 필요

➢ 예: integer 변수와 short integer 변수의 연산 등

예제 3-7

10진수 '21'과 '–21'에 대한 8-비트 길이의 부호화-크기 표현을 16-비트 길이로 확장하라.

풀이

+21= 00010101 (8-비트 부호화-크기 표현)

+21= 0000000000010101 (16-비트 부호화-크기 표현)

−21= 10010101 (8-비트 부호화-크기 표현)

−21= 1000000000010101 (16-비트 부호화-크기 표현)

참고:

CPU Word별 C 언어에서의 변수 크기. 컴파일러에 따라 다를 수 있음

자료형	16비트 CPU (예: DOS)	32비트 CPU (예: Windows XP, Linux)	64비트 CPU (예: Windows 10/11, Linux)
char	1 바이트	1 바이트	1 바이트
short	2 바이트	2 바이트	2 바이트
int	2 바이트	4 바이트	4 바이트
long	4 바이트	4 바이트	8 바이트 (대부분의 Unix/Linux 계열) 또는 4 바이트 (Windows)
long long	8 바이트	8 바이트	8 바이트
float	4 바이트	4 바이트	4 바이트
double	8 바이트	8 바이트	8 바이트

3.2.3 부호-비트 확장 (Bit Extension)

P154~p161

■ 부호-비트 확장

✓ 2의 보수 표현의 경우

➢ MSB(부호 비트) 밑으로 추가되는 확장 비트들을 부호 비트와 같은 값으로 세트

예제 3-8

10진수 '21'과 '–21'에 대한 8-비트 길이의 2의 보수 표현을 16-비트 길이로 확장하라.

풀이

+21 = 00010101 (8-비트 2의 보수)
+21 = 0000000000010101 (16-비트 2의 보수)
−21 = 11101011 (8-비트 2의 보수)
−21 = 1111111111101011 (16-비트 2의 보수)

3.3 논리 연산

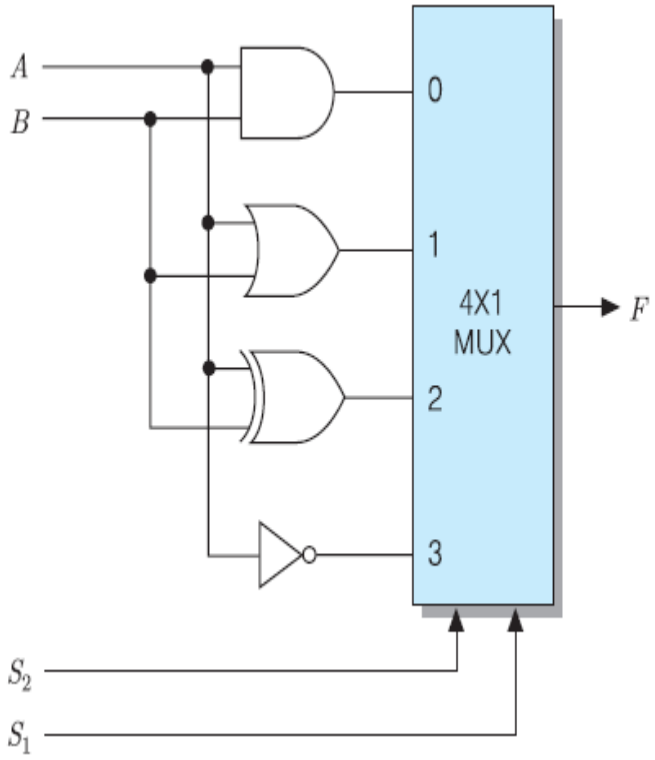
P162~p170

■ 논리 연산을 위한 하드웨어 모듈

✓ 하드웨어의 구성

- ➢ 입력 비트들은 모든 논리 게이트들을 통과
- ➢ 선택 신호들에 의하여 멀티플렉서의 네 입력들 중의 하나를 출력

A B	NOT A	NOT B	A AND B	A OR B	A XOR B
0 0	1	1	0	0	0
0 1	1	0	0	1	1
1 0	0	1	0	1	1
1 1	0	0	1	1	0



S_2 S_1	출력	연산
0 0	$F=A\wedge B$	AND
0 1	$F=A\vee B$	OR
1 0	$F=A\oplus B$	XOR
1 1	$F=\overline{A}$	NOT

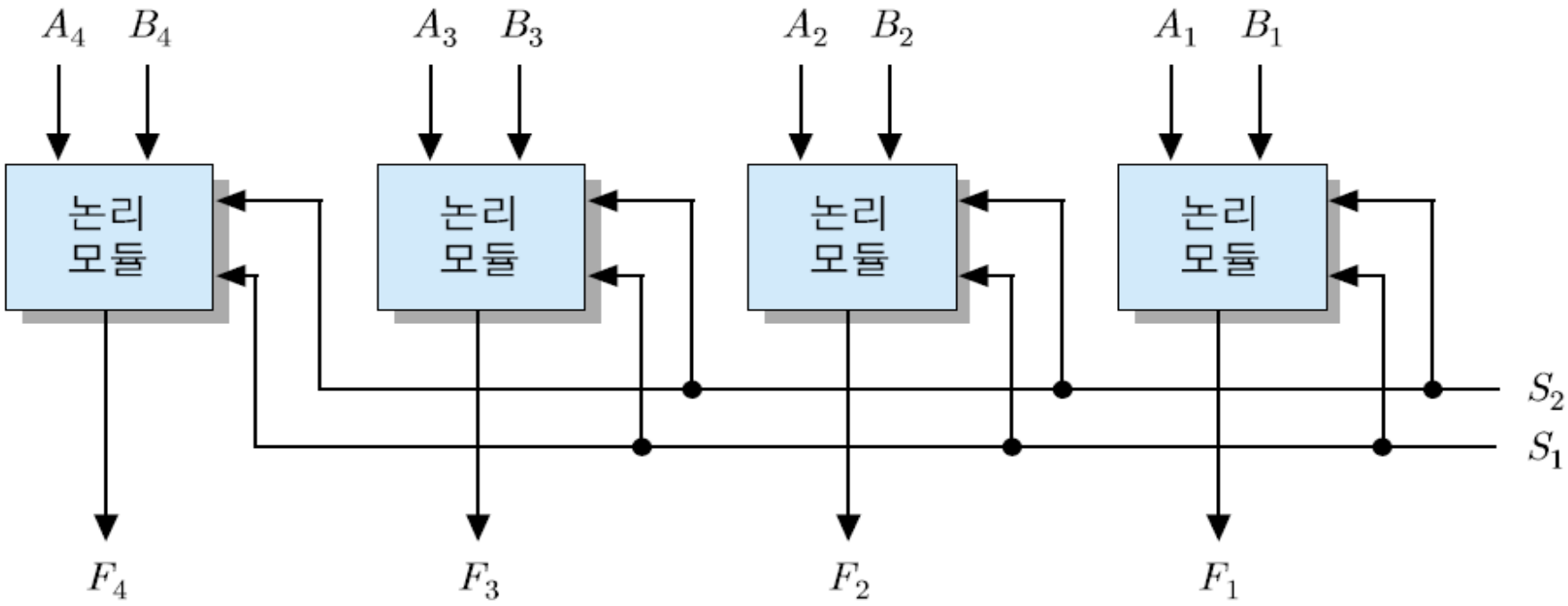
P162~p170

■ N-비트 논리 연산장치

✓ N-비트 데이터들을 위한 논리 연산장치

➢ 1비트 논리 모듈을 N 개로 연결

➢ [예] 4-비트 논리 연산장치



3.3 논리 연산

P162~p170

■ AND 연산 / OR 연산

✓ AND 연산

➢ 두 데이터 단어들의 대응되는 비트들 간에 AND 연산을 수행

```
A  =  1 0 1 1 0 1 0 1b
B  =  0 0 1 1 1 0 1 1b
--------------------------
      0 0 1 1 0 0 0 1b    (연산 결과)
```

✓ OR 연산

➢ 두 데이터 단어들의 대응되는 비트들 간에 OR 연산 수행

```
A  =  1 0 0 1 0 1 0 1b
B  =  0 0 1 1 1 0 1 1b
-------------------------
      1 0 1 1 1 1 1 1b   (연산 결과)
```

3.3 논리 연산

P162~p170

■ XOR 연산 / NOT 연산

✓ XOR 연산

➢ 두 데이터 단어들의 대응되는 비트들 간에 exclusive-OR 연산을 수행

```
A  =  1 0 0 1 0 1 0 1b
B  =  0 0 1 1 1 0 1 1b
------------------------
      1 0 1 0 1 1 1 0b    (연산 결과)
```

✓ NOT 연산

➢ 데이터 단어의 모든 비트들을 반전(invert)

```
A  =  1 0 0 1 0 1 0 1b    (연산 전)
------------------------------
      0 1 1 0 1 0 1 0b    (연산 결과)
```

3.3 논리 연산

P162~p170

■ 마스크(mask) 연산

- ✓ B 레지스터의 비트들 중에서 값이 0인 비트들과 같은 위치에 있는 A 레지스터의 비트들을 0으로 바꾸는(clear하는) 연산 → **특정 비트들 만을 0으로 clear하는 연산**
- ✓ AND 연산 이용
- ✓ 단어 내의 원하는 비트들을 선택적으로 clear하는 데 사용

```
A  =  1 1 0 1 0 1 0 1b    (연산 전)
B  =  0 0 0 0 1 1 1 1b
-------------------------
A  =  0 0 0 0 0 1 0 1b    (연산 결과)
```

3.3 논리 연산

P162~p170

■ 삽입(insert) 연산

✓ 새로운 비트 값들을 데이터 단어내의 특정 위치에 삽입

➢ **특정 비트들 만을 0, 1의 원하는 비트로 설정하는 연산**

✓ 방법

① 삽입할 비트 위치들에 대하여 마스크(AND) 연산 수행

② 새로이 삽입할 비트들과 OR 연산을 수행

✓ [예]

```
A  =  1 0 0 1 0 1 0 1b
B  =  0 0 0 0 1 1 1 1b    마스크 (AND 연산)
-----------------------
A  =  0 0 0 0 0 1 0 1b    첫 단계 결과
B  =  1 1 1 0 0 0 0 0b    삽입 (OR 연산)
-----------------------
A  =  1 1 1 0 0 1 0 1b    최종(삽입) 결과
```

P162~p170

■ 비교(compare) 연산

✓ A와 B 레지스터의 내용을 비교 → **XOR 연산**

- 만약 대응되는 비트들의 값이 같으면, A 레지스터의 해당 비트를 '0'으로 세트
- 만약 서로 다르면, A 레지스터의 해당 비트를 '1'로 세트
- 모든 비트들이 같으면(A = 00000000), 상태 레지스터의 Z(zero) 플래그를 1로 세트
- [예]

A = 1 1 0 1 0 1 0 1b

B = 1 0 0 1 0 1 1 0b

A = 0 1 0 0 0 0 1 1b (연산 결과)

3.4 시프트(shift) 연산

P170~p176

■ 논리적 시프트 (logical shift)

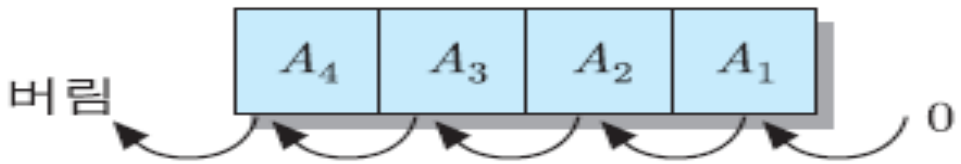
✓ 레지스터 내의 데이터 비트들을 왼쪽 혹은 오른쪽으로 한 칸씩 이동

논리적 시프트 → 숫자가 아니므로 그냥 비트들을 이동
산술적 시프트 → 숫자이므로 MSB를 주의하며 비트들을 이동
두 방식은 처리 방식도 다르며 CPU 명령어도 다름

✓ 좌측 시프트(left shift)

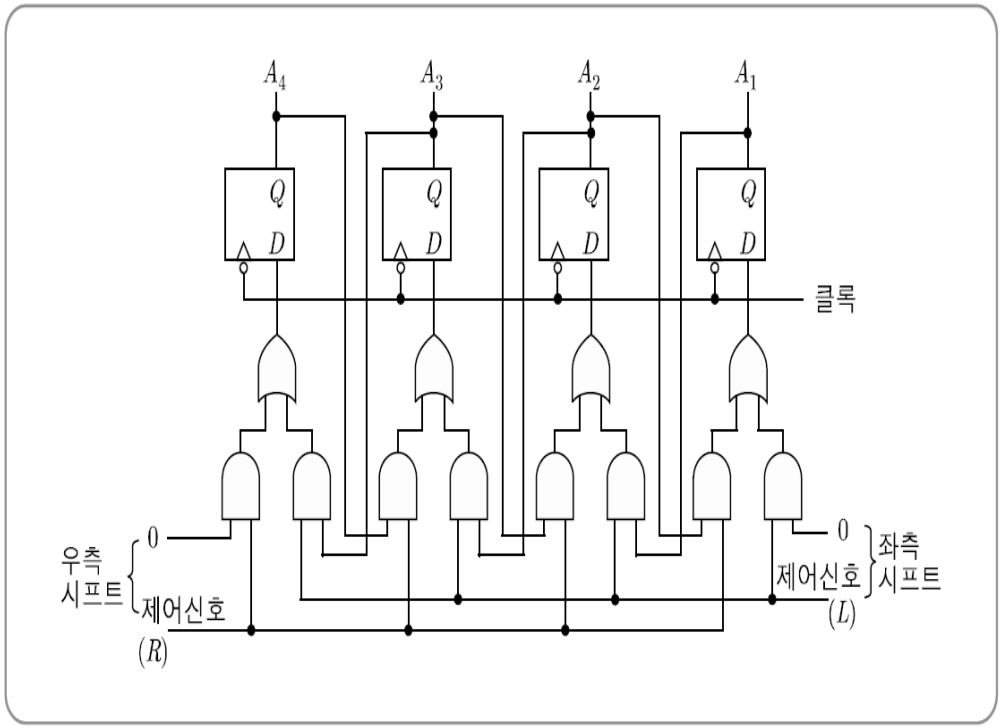
- 모든 비트들을 좌측으로 한 칸씩 이동 → ~~곱하기 2의 효과~~
- 최하위 비트(A_1)로는 '0'이 들어오고, 최상위 비트(A_4)는 버림

$$A_4 \leftarrow A_3,\ A_3 \leftarrow A_2,\ A_2 \leftarrow A_1,\ A_1 \leftarrow 0$$



✓ 우측 시프트(right shift)

- 모든 비트들이 우측으로 한 칸씩 이동 → ~~나누기 2의 효과~~
- 최상위 비트(A_4)로 '0'이 들어오고, 최하위 비트(A_0)는 버림

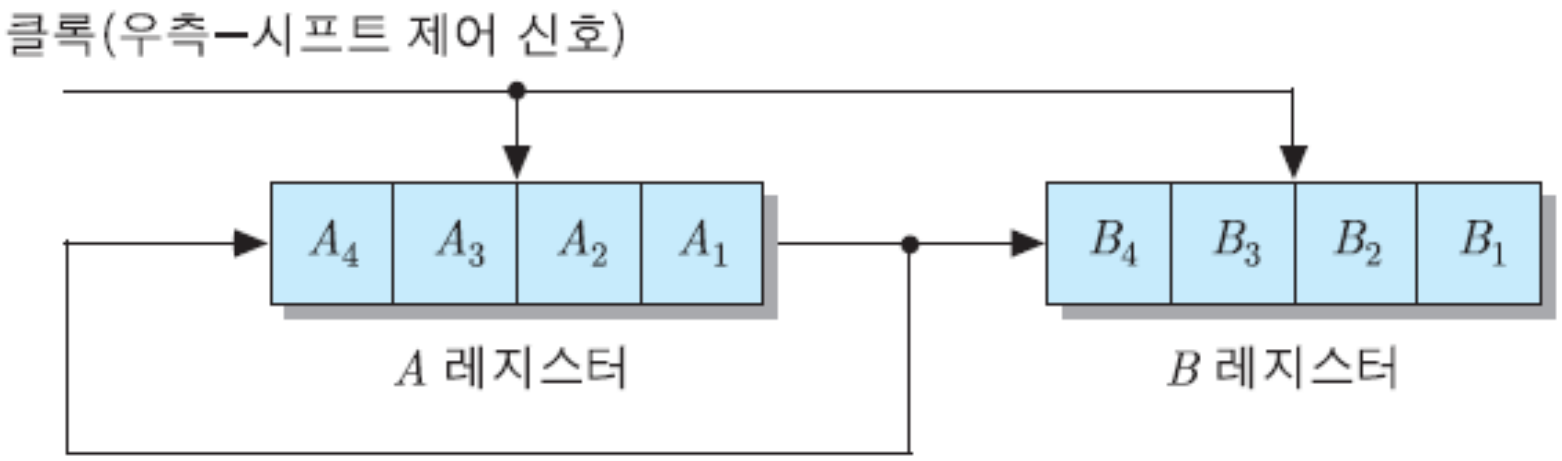


3.4 시프트(shift) 연산

P170~p176

■ 직렬 데이터 전송 (serial data transfer)

- ✓ 시프트 연산을 데이터 비트 수만큼 연속적으로 수행함으로써 두 레지스터들 사이에 한 개의 선을 통하여 전체 데이터를 이동하는 동작
- ✓ 직렬통신 방식(USART: Universal Synchronous/Asynchronous Receiver/Transmitter)의 송수신부에 많이 사용
  - ➢ 송신부: 병렬 입력 → 직렬 출력, 수신부: 직렬입력 → 병렬 출력



3.4 시프트(shift) 연산

P170~p176

■ 직렬 데이터 전송 예제

예제 3–18

그림 3–7과 같이 두 개의 4-비트 레지스터들이 직렬로 접속된 상태에서 *A* 레지스터에 저장되어 있는 데이터인 '1011'을 네 클록 주기 후에 모두 *B* 레지스터로 전송하고, *A* 레지스터는 원래의 데이터를 그대로 유지하게 되는 과정을 표시하라.

풀이

	A_4	A_3	A_2	A_1	B_4	B_3	B_2	B_1
초기상태	1	0	1	1	0	0	0	0
t_1	1	1	0	1	1	0	0	0
t_2	1	1	1	0	1	1	0	0
t_3	0	1	1	1	0	1	1	0
t_4	1	0	1	1	1	0	1	1

3.4 시프트(shift) 연산

P170~p176

■ 산술적 시프트(arithmetic shift)

- ✓ 수(number)를 나타내는 데이터에 대한 시프트
  - ➢ 시프트 과정에서 **부호 비트는 그대로 유지**시키고, 수의 크기를 나타내는 비트들만 시프트
- ✓ 산술적 좌측-시프트(arithmetic shift-left) → 곱하기 2의 효과
  - ➢ A_4 (불변), $A_3 \leftarrow A_2$, $A_2 \leftarrow A_1$, $A_1 \leftarrow 0$
- ✓ 산술적 우측-시프트(arithmetic shift-right) → 나누기 2의 효과
  - ➢ A_4 (불변), $A_4 \rightarrow A_3$, $A_3 \rightarrow A_2$, $A_2 \rightarrow A_1$

논리적 시프트 → 숫자가 아니므로 그냥 비트들을 이동
산술적 시프트 → 숫자이므로 MSB를 주의하며 비트들을 이동
두 방식은 처리 방식도 다르며 CPU 명령어도 다름

예제 3–19

10진수 –2에 대한 4-비트 2진수에 대하여 산술적 좌측-시프트를 수행한 후, 그 결과 값에 대하여 두 번의 산술적 우측-시프트를 수행하라.

풀이

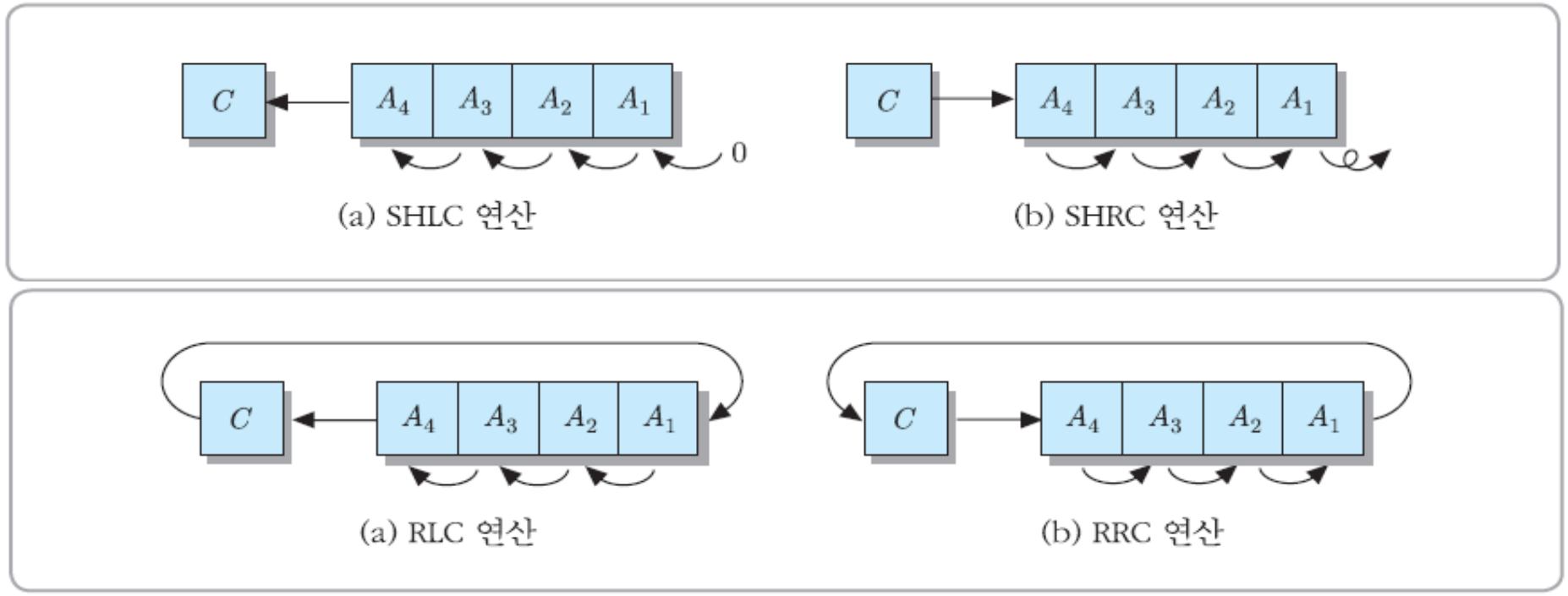
$A=$	1 1 1 0 (−2)	(초기 상태)
	1 1 0 0 (−4)	(산술적 좌측−시프트 결과)
	1 1 1 0 (−2)	(첫 번째 산술적 우측−시프트 결과)
	1 1 1 1 (−1)	(두 번째 산술적 우측−시프트 결과)

3.4 시프트(shift) 연산

P170~p176

■ C(Carry) 플래그를 포함한 시프트/순환 시프트 연산

- ✓ SHLC(shift left with carry): C 플래그를 포함한 좌측-시프트
- ✓ SHRC(shift right with carry): C 플래그를 포함한 우측-시프트
- ✓ RLC(rotate left with carry): C 플래그를 포함하는 좌측 순환 시프트(회전) 연산
- ✓ RRC(rotate right with carry): C 플래그를 포함하는 우측 순환 시프트(회전) 연산



3.5 정수의 산술 연산

P176~p192

■ 기본적인 산술 연산들

$A \leftarrow \overline{A} + 1$; 보수화(2의 보수 변환)
$A \leftarrow A + B$; 덧셈
$A \leftarrow A - B$; 뺄셈
$A \leftarrow A \times B$; 곱셈
$A \leftarrow A \div B$; 나눗셈
$A \leftarrow A + 1$; 증가(increment)
$A \leftarrow A - 1$; 감소(decrement)

3.5 정수의 산술 연산

■ 이진수 (유부호) 정수의 덧셈, 뺄셈 정리

요약자료 : 매우 중요

- ✓ **유부호(Signed) 정수 표현 방법**
  - ➢ **①부호화 크기, ②1의 보수, ③2의 보수** → 장단점 및 수 표현범위
- ✓ 1의 보수, 2의 보수 이진수의 뺄셈 → 보수로 만들어 덧셈 (우리가 수체계를 그렇게 정의했다)
  - ➢ 1의 보수에서 반대 부호 값 얻는 방법 → 모든 이진수를 0 ↔ 1 (자릿수 조심)
  - ➢ 2의 보수에서 반대 부호 값 얻는 방법 → 모든 이진수를 0 ↔ 1 하고 1을 더함 (자릿수 조심)
- ✓ **오버플로우**
  - ➢ 유부호 정수의 덧셈/뺄셈 결과 값이 표현범위를 초과한 경우 → 오류
  - ➢ 아랫자리에서 MSB로 입력되는 캐리와 MSB에서 윗자리로 나가는 캐리의 값이 다를 때 발생 → 쉬운 판별법
- ✓ 오버플로우가 아니면서 MSB에서 윗자리로 나가는 캐리의 발생시 처리 방법
  - ➢ 2의 보수 계산에서는 이를 무시
  - ➢ 1의 보수 계산에서는 이를 가장 낮은 자리에 다시 더해준다

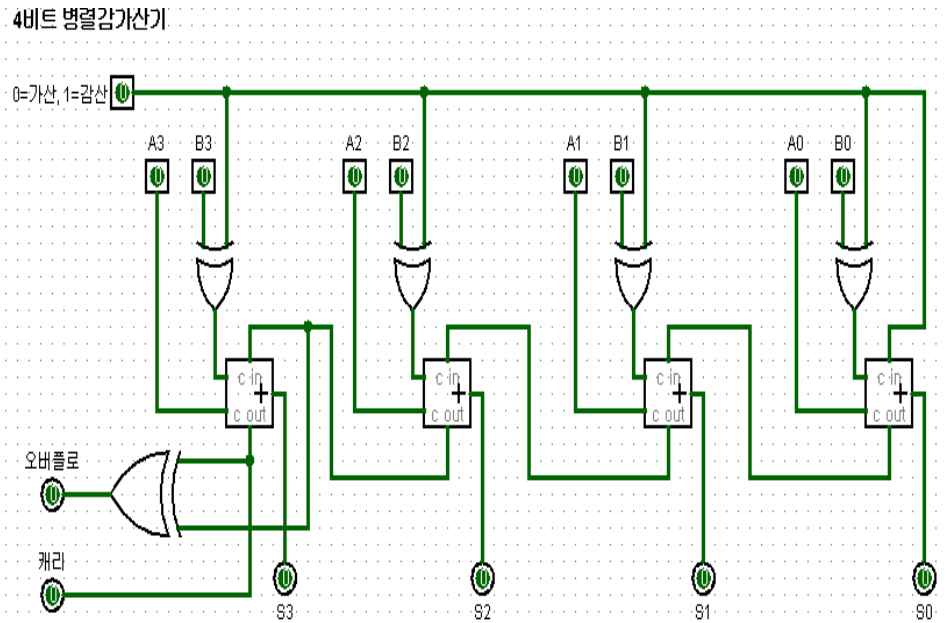
3.5 정수의 산술 연산

■ 추가: 이진수 정수의 덧셈, 뺄셈 정리

교재에 없는 무부호 정수에 대한 덧셈, 뺄셈 내용 추가

- ✓ ALU는 이진수 정수의 덧셈, 뺄셈하는 경우 유부호, 무부호를 구별하지 않고 **오직 하나의 덧셈 연산회로**만을 사용
- ✓ 결과가 나오면 컴파일러가 유부호, 무부호에 따라 CF(Carry Flag), OF(Overflow Flag) 비트를 이용해 연산의 오류를 판단
  - ➢ 컴파일러가 **유부호 연산**을 한 경우 **OF 비트**로만 오류를 판단, CF는 무시
  - ➢ 컴파일러가 **무부호 연산**을 한 경우 **CF 비트**로만 오류를 판단, OF는 무시

구분	CF (Carry Flag)	OF (Overflow Flag)
관련 수 체계	**부호 없는 수 (Unsigned)**	**부호 있는 수 (Signed)**
의미	MSB의 자리올림(carry) 또는 빌림(borrow) 발생 여부	연산 결과의 부호가 변하는 넘침(overflow) 발생 여부
언제 확인하는가?	부호 없는 수(무부호 정수) 연산 후	부호 있는 수(유부호 정수) 연산 후
1'일 때의 해석	덧셈: 결과가 범위를 초과함 → 오류 뺄셈: 빌림이 발생하지 않음 → 정상	결과가 범위를 초과하여 부호가 잘못됨 → 오류
0'일 때의 해석	덧셈: 결과가 범위 내에 있음 → 정상 뺄셈: 빌림이 발생함 → 오류	결과가 정상 범위 내에 있음 → 정상



3.5.1 덧셈

이 부분부터는 2의 보수 체계만을 사용

P176~p192

■ 2의 보수로 표현된 수들의 덧셈 방법

✓ 두 수를 더하고, 만약 MSB에서 올림수(캐리)가 발생하면 버림

➢ 1의 보수 계산에서는 캐리를 가장 낮은 자리에 한번 더 더해준다

예제 3–20

아래와 같은 2진수 덧셈들을 수행하고, 10진수 덧셈의 결과와 비교하라.

풀이

(a) $(+3)+(+4) = +7$

$$\begin{array}{r} 0011 \\ +\ 0100 \\ \hline 0111 \end{array} = +7$$

(b) $(-3)+(+3) = 0$

$$\begin{array}{r} 1101 \\ +\ 0011 \\ \hline \textcircled{1}\ 0000 \end{array} = 0$$

버림

(c) $(-6)+(+2) = -4$

$$\begin{array}{r} 1010 \\ +\ 0010 \\ \hline 1100 \end{array} = -4$$

(d) $(-4)+(-1) = -5$

$$\begin{array}{r} 11111100 \\ +\ 11111111 \\ \hline \textcircled{1}\ 11111011 \end{array} = -5$$

3.5.1 덧셈

P176~p192

■ 반가산기(Half Adder)

- 두 개의 입력과 두개의 출력(Sum, Carry)으로 구성되는 가장 기본적인 가산기

$$\begin{array}{r} A \\ +\ B \\ \hline C\ \ S \end{array} \qquad \begin{array}{r} 0 \\ +\ 0 \\ \hline 0\ \ 0 \end{array} \qquad \begin{array}{r} 0 \\ +\ 1 \\ \hline 0\ \ 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 0 \\ \hline 0\ \ 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 1 \\ \hline 1\ \ 0 \end{array}$$

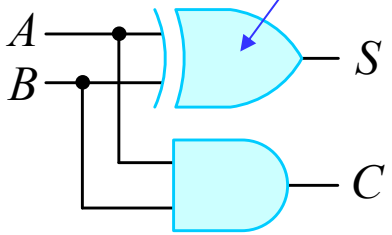
S : sum
C : carry

입력		출력	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

진리표

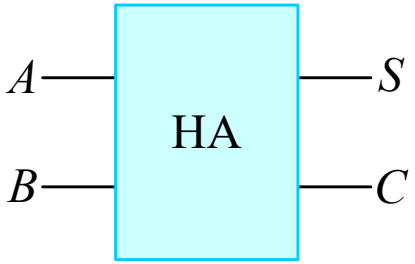
$$S = \overline{A}B + A\overline{B} = A \oplus B$$
$$C = A \cdot B$$

논리식



논리회로

XOR(익스클루시브 오어) 게이트는 입력의 값이 서로 다를 때 출력이 1이 됨



3.5.1 덧셈

P176~p192

■ 전가산기(Full Adder)

✓ 자리 올림수(carry)를 고려하여 만든 덧셈 회로 (캐리 입력과 캐리 출력 기능)

C_{in}	0	1	0	1	0	1	0	1
A	0	0	1	1	0	0	1	1
$+\ B$	$+\ 0$	$+\ 0$	$+\ 0$	$+\ 0$	$+\ 1$	$+\ 1$	$+\ 1$	$+\ 1$
$C_{out}S$	0 0	0 1	0 1	1 0	0 1	1 0	1 0	1 1

입력			출력	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$
\begin{aligned}
S &= \overline{A}\,\overline{B}C_{in} + \overline{A}B\overline{C_{in}} + A\overline{B}\,\overline{C_{in}} + ABC_{in} \\
&= \overline{A}(\overline{B}C_{in} + B\overline{C_{in}}) + A(\overline{B}\,\overline{C_{in}} + BC_{in}) \\
&= \overline{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) \\
&= A \oplus (B \oplus C_{in}) = (A \oplus B) \oplus C_{in}
\end{aligned}
$$

$$
\begin{aligned}
C_{out} &= \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C_{in}} + ABC_{in} \\
&= C_{in}(\overline{A}B + A\overline{B}) + AB(\overline{C_{in}} + C_{in}) \\
&= C_{in}(A \oplus B) + AB
\end{aligned}
$$

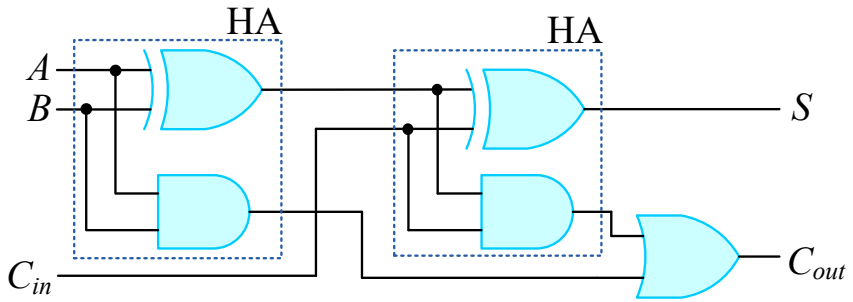
진리표와 논리식

3.5.1 덧셈

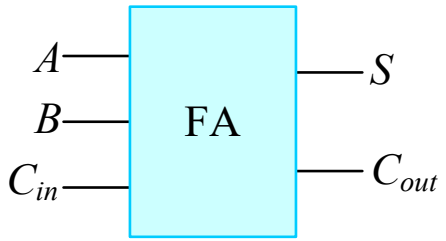
P176~p192

■ 전가산기(Full Adder)

✓ 전가산기는 반가산기 2개와 OR 게이트를 이용하여 구성



논리회로



논리기호

$$S = A \oplus B \oplus C_{in}$$

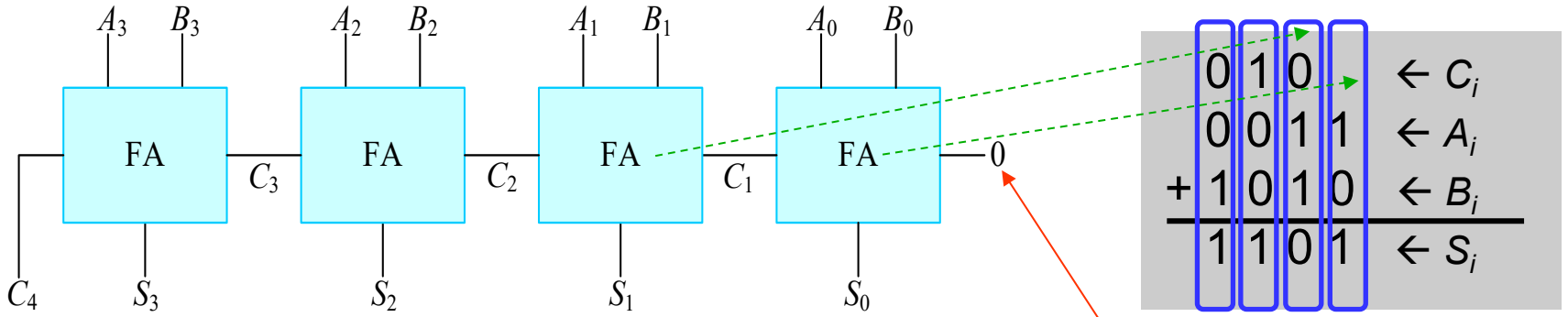
$$C_{out} = C_{in}(A \oplus B) + AB$$

3.5.1 덧셈

P176~p192

■ 병렬 가산기(parallel adder)

- ✓ **전가산기(1비트) 여러 개를 병렬로 연결**한 회로
- ✓ **리플-캐리 가산기(ripple-carry adder)**라고도 함.
  - ➢ 복수개의 비트 입력들의 연산(가산)이 가능하다
  - ➢ 아랫단에서 계산이 끝난 후에 발생되는 캐리 출력을 윗단의 캐리 입력으로 사용하기 때문에 단수가 늘어나면 **속도가 매우 느려진다**



전가산기를 이용한 병렬가산기

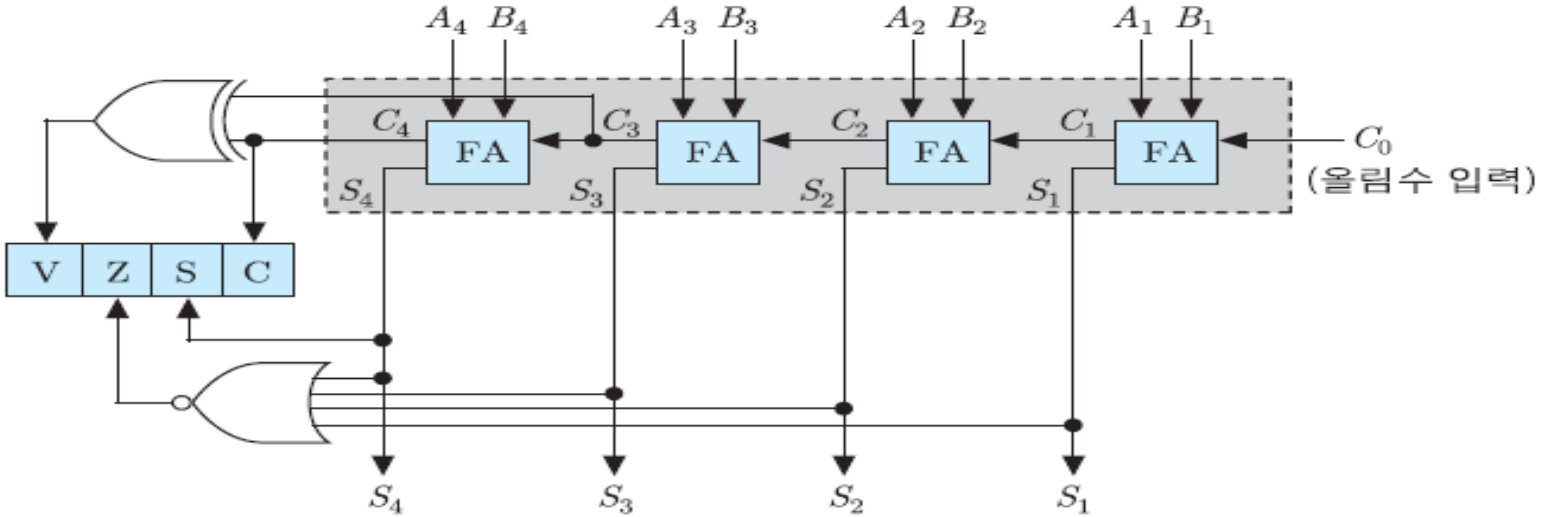
반가산기 사용 가능

3.5.1 덧셈

P176~p192

■ 병렬 가산기(parallel adder)

- ✓ 덧셈을 수행하는 하드웨어 모듈로 비트 수만큼의 전가산기(full-adder)들로 구성
- ✓ 덧셈 연산 결과에 따라 해당 조건 플래그들(condition flags)을 세트
  - ➢ C 플래그: 올림수(carry)
  - ➢ S 플래그: 부호(sign)
  - ➢ Z 플래그: 0(zero)
  - ➢ V 플래그: 오버플로우(overflow)
- ✓ 4-비트 병렬 가산기와 상태 비트 제어회로



3.5.1 덧셈

■ 참고: 캐리 예견(미리보기) 가산기

자릿수와 상관없이 전체 연산시간은 동일

- ✓ CLA : Carry-Look ahead Adder
- ✓ 필요성
  - ➢ 리플 캐리 가산기에서 지연의 원인이 되는 캐리의 입력 방법 개선
    - ❖ **비트(단) 수와 상관 없이 캐리를 일찍(한번에) 알아낼 수 있는 회로의 설계 검토**
- ✓ 원리
  - ➢ 모든 입력 캐리 값들은 입력 데이터로부터 직접 동시에 추출
  - ➢ 모든 비트(단)의 캐리를 동시에 한번에 계산하고 모든 비트의 합도 병렬적으로 처리
- ✓ 장단점
  - ➢ 지연의 최소화로 인한 **속도의 개선**
  - ➢ 회로의 복잡
- ✓ 제한점
  - ➢ Fan-In

■ 참고: 캐리 예견(미리보기) 가산기 - 계속

✓ CLA는 원리는 Ai, Bi 모두가 1일 때, 또는 Ai, Bi 둘 중에 하나가 1이고 Ci 가 1일 때 캐리가 발생하므로 특정단에서의 캐리 출력 논리식은 다음과 같다.

$$C_{out} = C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i = A_iB_i + (A_i + B_i)C_i = G_i + P_iC_i$$

where $G_i = A_iB_i$ $P_i = A_i \oplus B_i$

G: generate
P: propagate

✓ 4비트 가산기에서 위 식을 써보면 다음과 같다

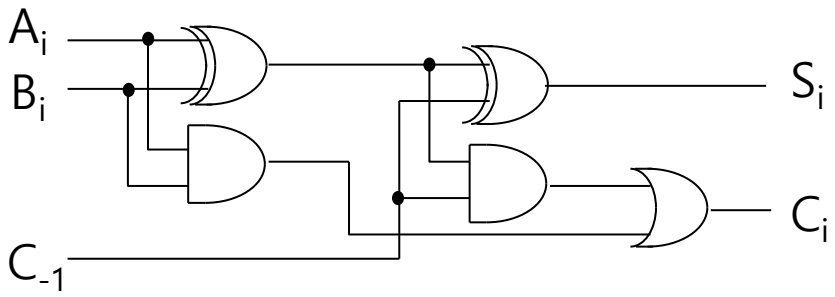
$$C_1 = G_0 + P_0C_0$$
$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$$
$$C_3 = G_2 + P_2C_2 = G_2 + P_2(G_1 + P_1G_0 + P_1P_0C_0) = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$
$$C_4 = G_3 + P_3C_3 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$
$$S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$$

✓ 상기 식에서 C_i는 이전 단의 C_i를 필요로 하지 않는다. 단지 C_0와 이전 단들의 G, P 만 알면 된다

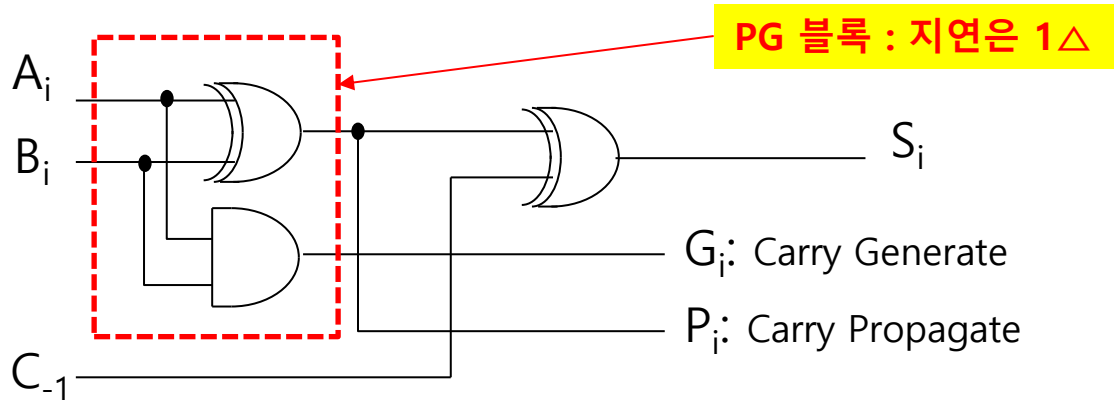
✓ P와 G도 A, B 입력으로 바로 계산된다.

■ 참고: 캐리 예견(미리보기) 가산기 - 계속

✓ 일반적인 리플캐리 1비트 전가산기



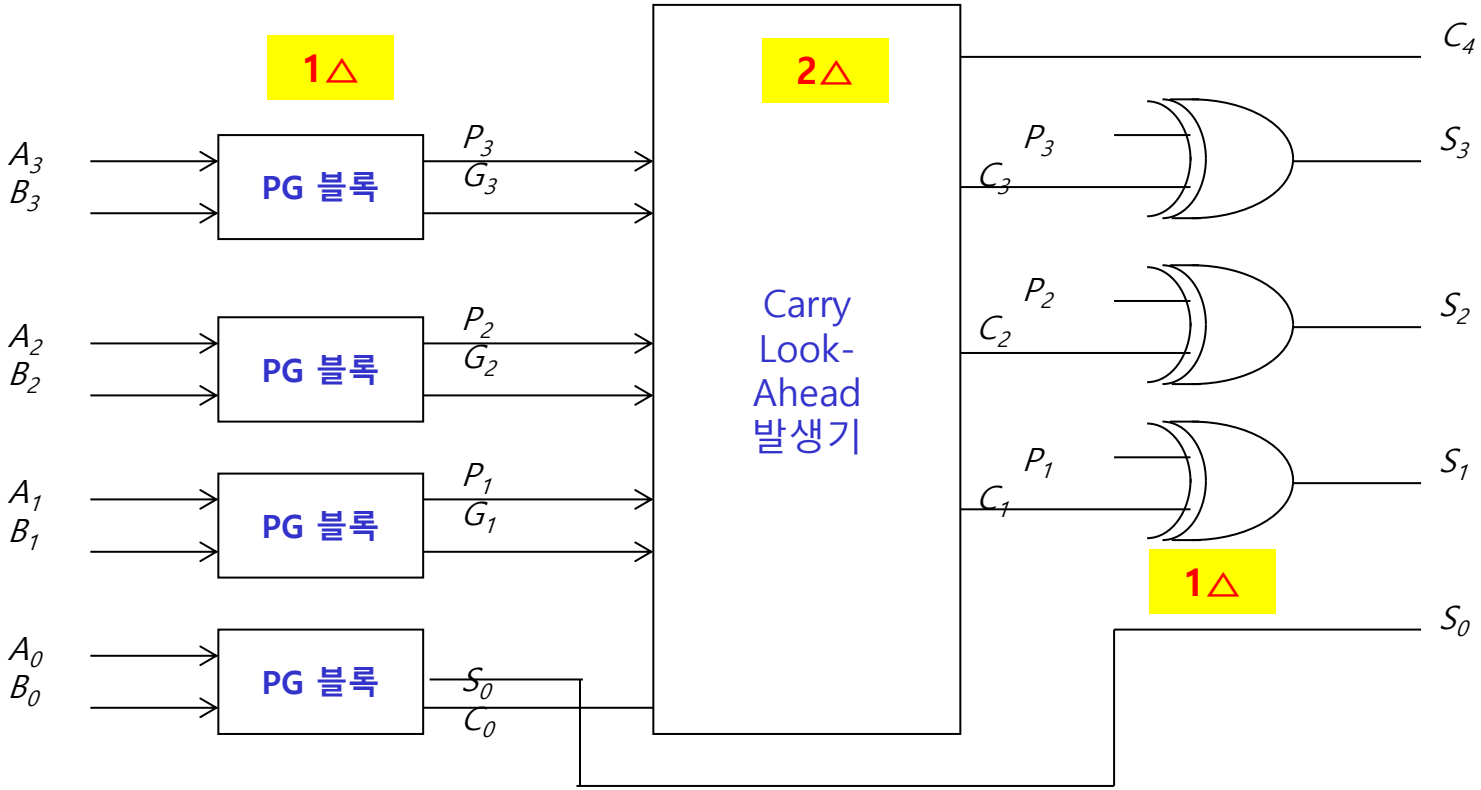
✓ P, G 개념을 도입한 PG 블록



3.5.1 덧셈

■ 참고: 캐리 예견(미리보기) 가산기 - 계속

자릿수와 상관없이 전체 지연은 항상 4△



<캐리 예견 가산기를 이용한 4 비트 병렬가산기>

3.5.1 덧셈

P176~p192

■ 덧셈 오버플로우

✓ 연산 결과가 **표현할 수 있는 범위를 초과**하여 결과값이 틀리게 되는 상태 → **오류**

✓ 검출 방법

➢ 두 올림수(carry)들 간의 exclusive-OR를 이용

➢ $V = C_4 \oplus C_3$

예제 3–21

아래의 덧셈에서 오버플로우가 발생하는지 확인하라.

풀이

(a) $(+6)+(+3) = +9$

$$\begin{array}{r} 0110 \\ +\ 0011 \\ \hline 1001 \end{array} = -7 \text{ (오버플로우)}$$

(b) $(-7)+(-6) = -13$

$$\begin{array}{r} 1001 \\ +\ 1010 \\ \hline \textcircled{1}\ 0011 \end{array} = +3 \text{ (오버플로우)}$$

3.5.2 뺄셈

P176~p192

■ 덧셈을 이용하여 수행

- ✓ A : 피감수(minuend), B : 감수(subtrahend)
- ✓ **A - (+B) = A + (-B)**
- ✓ **A - (-B) = A + (+B)**

예제 3–22

2의 보수를 이용하여 아래와 같은 2진수 뺄셈들을 수행하라.

풀이

(a) $(+2)-(+6) = (+2)+(-6) = -4$

$$\begin{array}{r} 0010 \\ +\ 1010 \\ \hline 1100 \end{array} = -4$$

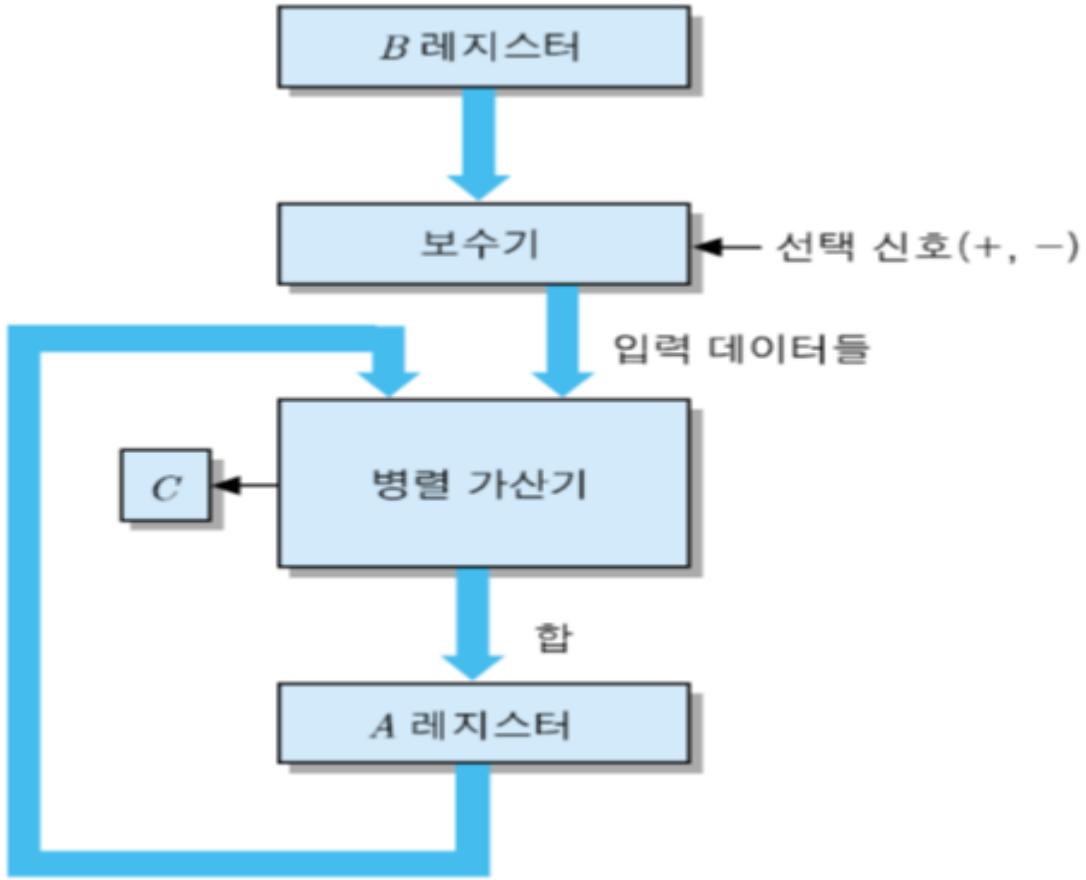
(b) $(+7)-(+4) = (+7)+(-4) = +3$

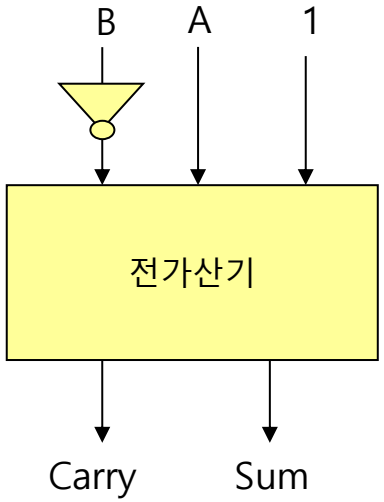
$$\begin{array}{r} 00000111 \\ +\ 11111100 \\ \hline \textcircled{1}00000011 \end{array} = +3$$

3.5.2 뺄셈

P176~p192

■ 덧셈과 뺄셈 겸용 하드웨어의 블록 구성도(2의 보수)



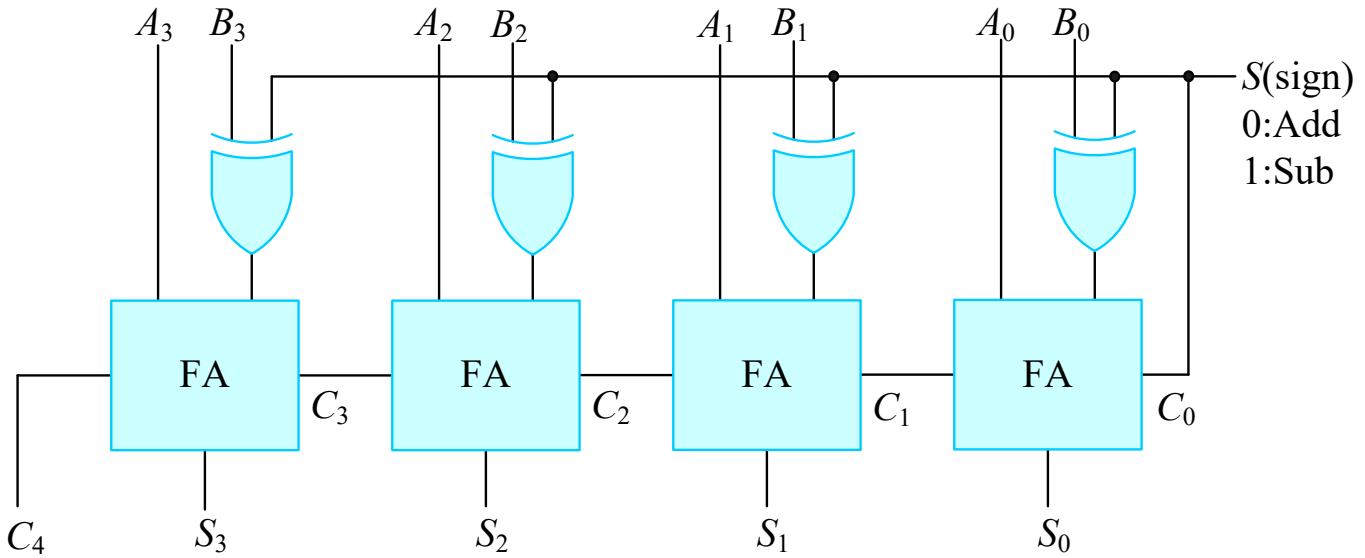


3.5.2 뺄셈

P176~p192

■ 병렬 가감산기 (parallel adder-subtracter)

- ✓ 병렬가산기의 B입력을 부호 S(sign)와 XOR하여 전가산기의 입력으로 사용함으로써 덧셈과 뺄셈이 모두 가능한 회로
- ✓ 2의 보수 이용
  - ➢ A - B = A + (-B)



병렬가감산기

S	B	F
0	0	0
0	1	1
1	0	1
1	1	0

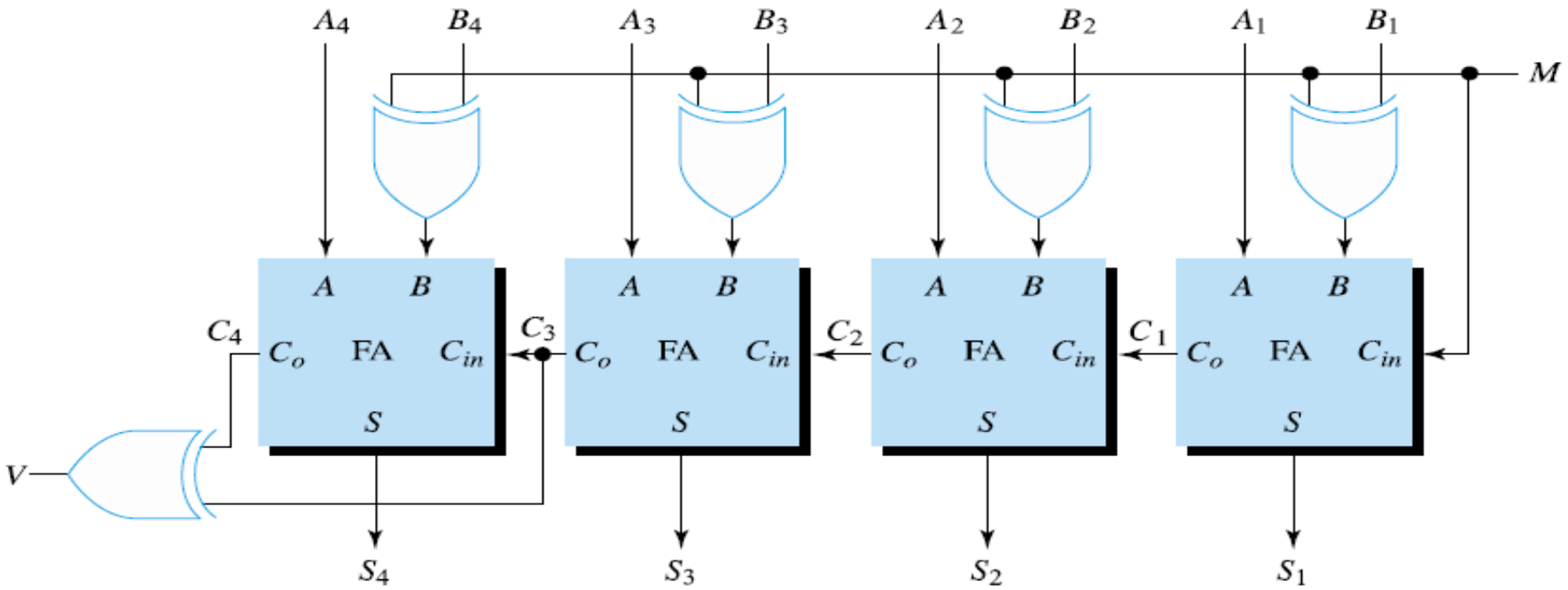
3.5.2 뺄셈

P176~p192

■ 4-비트 병렬 가감산기 (4-bit parallel adder-subtracter)

✓ 4-비트 데이터들 간의 덧셈($A+B$) 및 뺄셈(A-B)을 모두 수행하는 조합회로

- ➢ 제어 신호 $M = 0$ → 덧셈
- ➢ 제어 신호 $M = 1$ → 뺄셈
  - ❖ 입력 B의 비트들을 반전하고, 최하위 올림수로서 M 을 입력 → 2의 보수 개념



S	B	F
0	0	0
0	1	1
1	0	1
1	1	0

P176~p192

■ 뺄셈 오버플로우

- ✓ 뺄셈 결과가 그 범위를 초과하여 결과값이 틀리게 되는 상태
- ✓ 검출 방법 : 덧셈과 동일 ($V = C_4 \oplus C_3$)

예제 3–23

2의 보수를 이용하여 수행하는 아래의 뺄셈들에서 오버플로우가 발생하는지 검사하라.

풀이

(a) $(+6)-(-4) = (+6)+(+4) = +10$

$$\begin{array}{r} 0110 \\ +\ 0100 \\ \hline 1010 \end{array} = -6 \quad (C_4 \oplus C_3 = 0 \oplus 1 = 1 : \text{오버플로우 발생})$$

(b) $(-7)-(+6) = (-7)+(-6) = -13$

$$\begin{array}{r} 1001 \\ +\ 1010 \\ \hline \textcircled{1}\ 0011 \end{array} = +3 \quad (C_4 \oplus C_3 = 1 \oplus 0 = 1 : \text{오버플로우 발생})$$

3.5.3 곱셈

P176~p192

■ 부호 없는 정수의 곱셈

✓ 방법

① 각 비트에 대하여 **부분 적**(partial product) 계산

② 부분 적들을 모두 더하여 최종 결과를 얻음

예제 3-24

2진수 곱셈(1101×1011)을 수행하고, 결과값이 10진수 곱셈의 결과(13×11=143)와 일치하는지 확인하라.

풀이

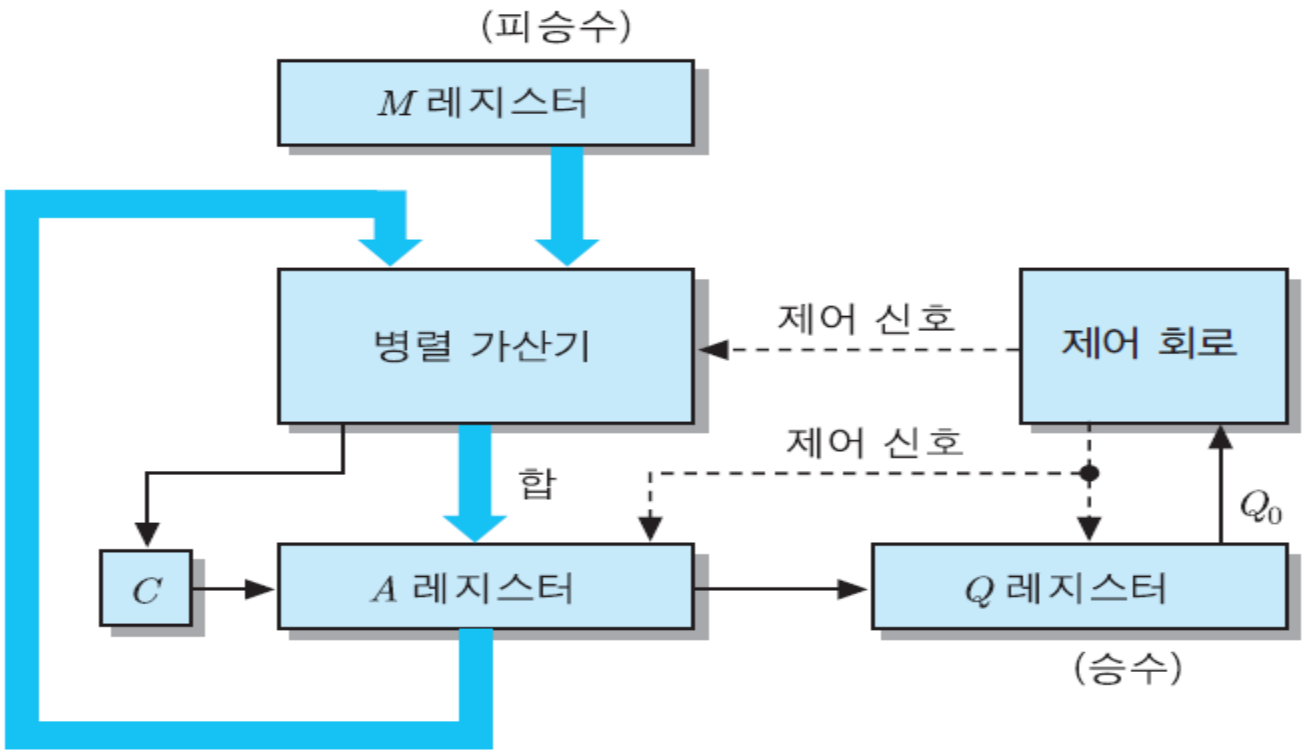
```
    1101        (피승수=13)
  × 1011        (승수=11)
  ------
    1101  }
   1101   }
  0000    }     부분 적들(partial products)
 1101     }
  ------
 10001111       (최종 결과=143)
```

3.5.3 곱셈

P176~p192

■ 부호 없는 정수 승산기의 하드웨어 구성도

- ✓ *M* 레지스터: 피승수(multiplicand) 저장
- ✓ *Q* 레지스터: 승수(multiplier) 저장
- ✓ 두 배 길이의 결과값은 *A* 레지스터와 *Q* 레지스터에 저장

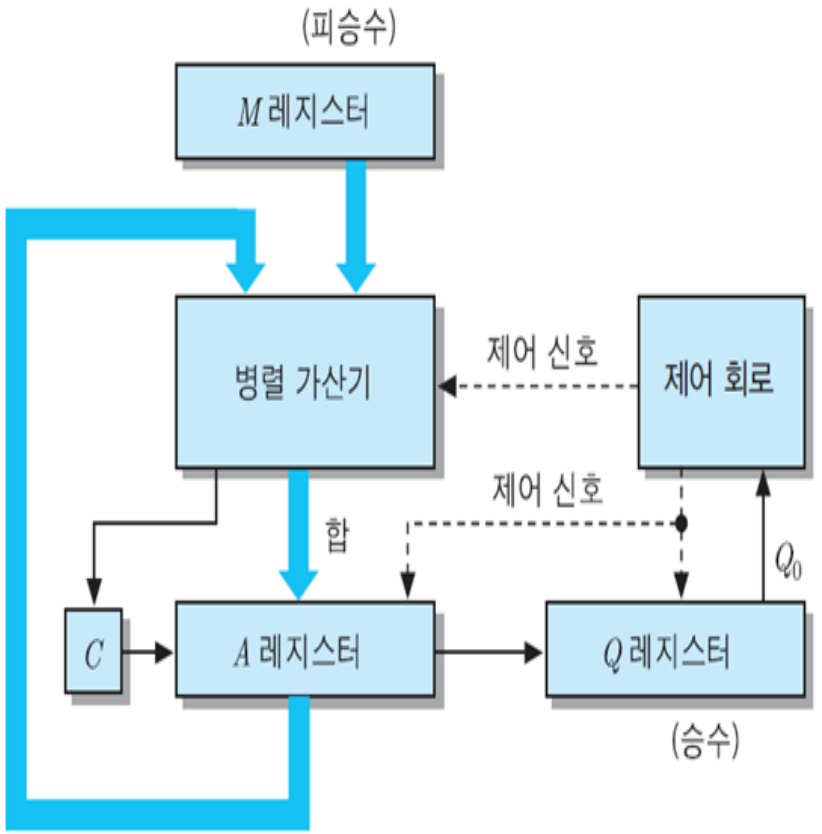


3.5.3 곱셈

P176~p192

- **부호 없는 정수 승산기의 곱셈이 수행되는 과정에서의 레지스터 내용들**

	C	A	Q	
[초기 상태]	0	0000	1011	
[사이클 1]	0	1101	1011	; Q_0=1이므로, $A \leftarrow A+M$
	0	0110	1101	; 우측 시프트($C-A-Q$)
[사이클 2]	1	0011	1101	; Q_0=1이므로, $A \leftarrow A+M$
	0	1001	1110	; 우측 시프트($C-A-Q$)
[사이클 3]	0	0100	1111	; Q_0=0이므로, 우측 시프트($C-A-Q$)만 수행
[사이클 4]	1	0001	1111	; Q_0=1이므로, $A \leftarrow A+M$
	0	1000	1111	; 우측 시프트($C-A-Q$)

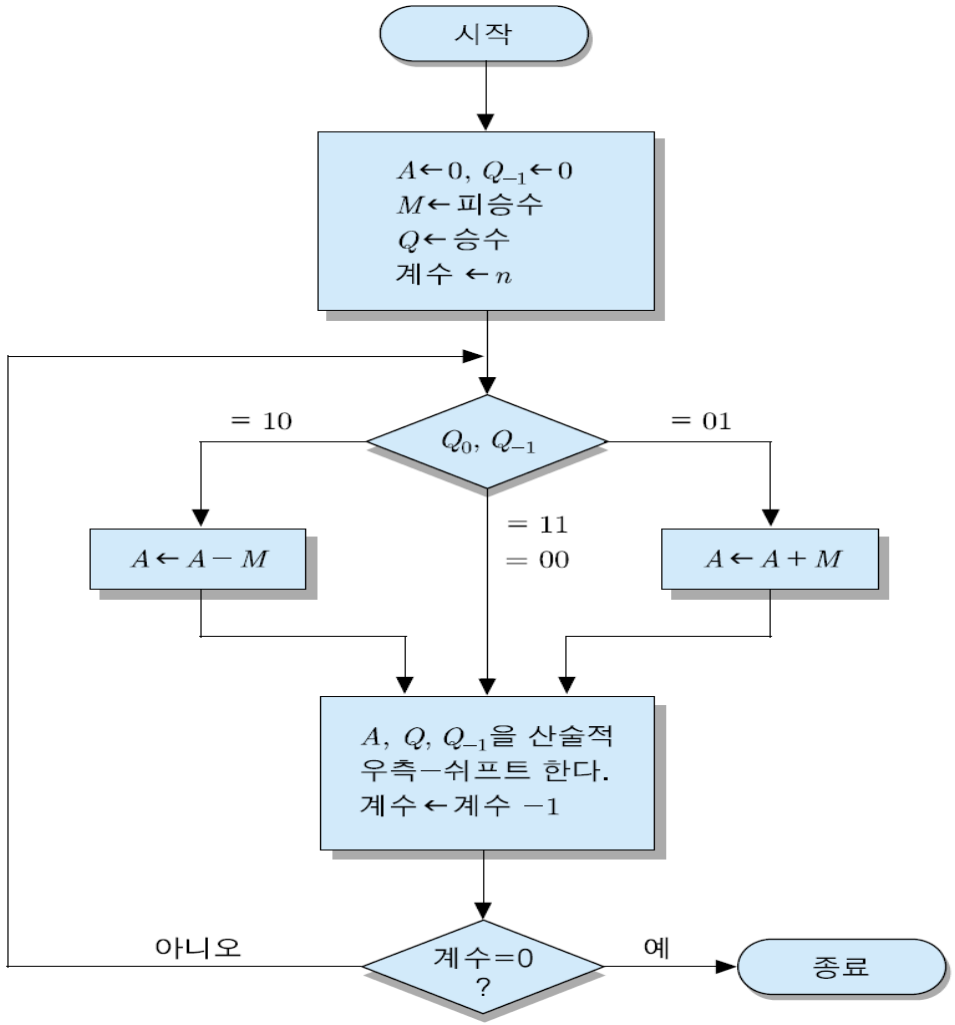


3.5.3 곱셈

P176~p192

■ 부호 있는 2의 보수들 간의 곱셈

- ✓ Booth 알고리즘(Booth's algorithm) 사용
- ✓ 하드웨어 구성
  - ➢ 부호 없는 정수 승산기의 하드웨어에 다음 부분을 추가
  - ➢ M 레지스터와 병렬 가산기 사이에 보수기(complementer) 추가
  - ➢ Q 레지스터의 우측에 Q_{-1} 이라고 부르는 1-비트 레지스터를 추가하고, 출력을 Q_0와 함께 제어 회로로 입력



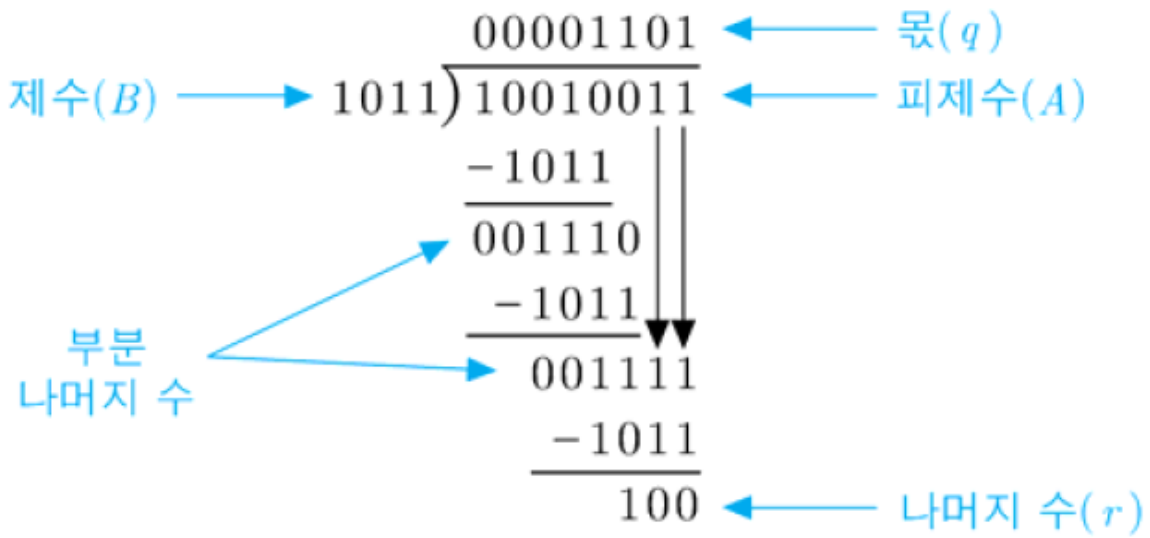
3.5.4 나눗셈

P176~p192

■ 나눗셈의 수식 표현

✓ 이진 곱셈보다는 복잡하지만 같은 원리

- 즉 반복적인 시프트와 덧셈 또는 뺄셈으로 구성
- $A \div B = q \cdots\cdots r$
- A: 피제수(dividend), B: 제수(divisor), q: 몫(quotient), r: 나머지 수(remainder)



3.6 부동소수점 수의 표현

P193~p204

■ 부동소수점 표현(floating-point representation)

✓ 소수점의 위치를 이동시킬 수 있는 수 표현 방법 → 수 표현 범위 확대

✓ 부동소수점 수(floating-point number)의 일반적인 형태

➢ $N = (-1)^S M \times B^E$

➢ S: 부호(sign), M: 가수(mantissa), B: 기수(base), E: 지수(exponent)

✓ 10진 부동소수점 수 → 기수 $B = 10$

➢ [예] 274,000,000,000,000 → 2.74×10^{14}

0.00000000000274 → 2.74×10^{-12}

✓ 2진 부동소수점 수 → 기수 $B = 2$

➢ [예] 11.101 → 0.11101×2^2, 0.00001101 → 0.1101×2^{-4}

➢ 단일-정밀도(single-precision) 2진 부동소수점 수 → 32 비트 구조

➢ 복수-정밀도(double-precision) 2진 부동소수점 수 → 64 비트 구조

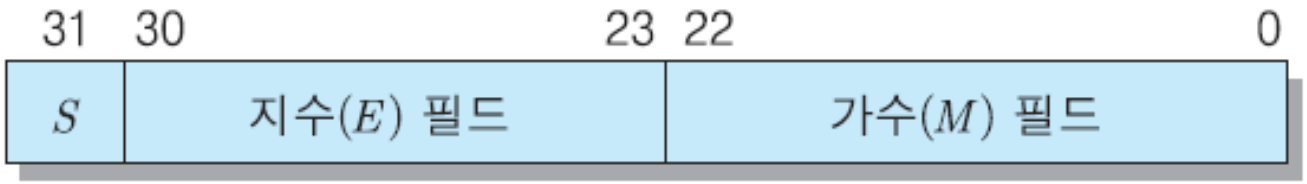
교재에서 설명하고 있는 이 부분은 부동소수점의 개념을 파악하기 위한 부분이며, 실제로 근래의 컴퓨터에서는 뒷부분에서 학습하는 IEEE 표준을 사용한다.
이 부분의 내용과 IEEE 표준의 내용은 정규화 위치 및 바이어스 부분을 제외하고는 동일함.
시험을 볼 때는 IEEE 방식으로 처리해야 정답으로 인정

3.6 부동소수점 수의 표현

P193~p204

■ 단일-정밀도 부동소수점 수 형식의 예

✓ *S*: 1 비트, *E*: 8 비트, *M*: 23 비트



✓ 지수(*E*) 필드의 비트 수 증가

➢ **표현 가능한 수의 범위 확장**

✓ 가수(*M*) 필드의 비트 수 증가

➢ **정밀도(precision) 증가**

✓ 표현가능한 수 크기의 범위

➢ $(0.5 \times 2^{-128} \sim 0.5 \times 2^{127}) \approx 1.47 \times 10^{-39} \sim 1.7 \times 10^{38}$

3.6 부동소수점 수의 표현

P193~p204

■ 같은 수에 대한 부동소수점 표현

✓ 같은 수에 대한 부동소수점 표현이 여러 가지가 존재

- 0.1101 × 2^5 ← 정규화된 표현
- 11.01 × 2^3
- 0.001101 × 2^7

✓ **정규화**된 표현(Normalized representation)

- 수에 대한 표현을 한 가지로 통일하기 위한 방법
  - ❖ $\pm 0.1bbb...b \times 2^E$
- 위의 예에서 정규화된 표현은 **0.1101×2^5**

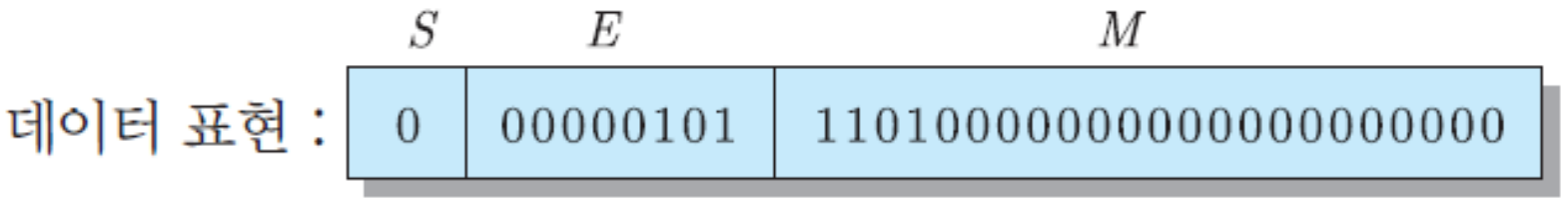
3.6 부동소수점 수의 표현

P193~p204

■ 부동소수점 표현의 예 (0.1101×2^5)

- ✓ 부호(S) 비트 = 0
- ✓ 지수(E) = 00000101
- ✓ 가수(M) = 1101 0000 0000 0000 0000 000

이 형식은 일반적이지 않은 방법이다.
뒷부분에 학습하는 히든 비트, 바이어스를 포함해야 실제 사용하는 부동소수점 형식임.
여기서는 부동소수점의 개념을 설명하기 위한 예로만 보면 됨

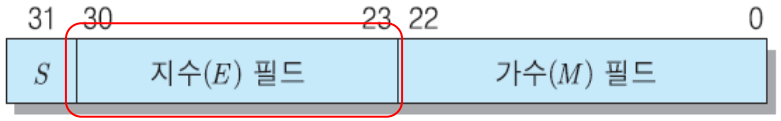


- ✓ 소수점 아래 첫 번째 비트는 항상 1이므로, 반드시 저장할 필요는 없음 → 히든(hidden) 비트
- ✓ 가수 23 비트를 이용하여 소수점 아래 24 자리 수까지 표현 가능

3.6 부동소수점 수의 표현

P193~p204

■ 바이어스된 지수(biased exponent)



- ✓ 지수를 바이어스된 수(biased number)로 표현
- ✓ **지수의 양수 및 음수 표현을 하기 위함 → 교재에는 없으나 이게 중요한 목적임**
  - ➢ 127 바이어스의 경우 → 지수필드가 127이면 0, 128이면 +1, 126이면 -1을 의미
  - ➢ 128 바이어스의 경우 → 지수필드가 128이면 0, 129이면 +1, 127이면 -1을 의미
- ✓ 8-비트 바이어스된 지수값들(8-bit biased exponents)

지수 비트 패턴	절대값	실제 지수값	
		바이어스 = 127	바이어스 = 128
11111111	255	+128	+127
11111110	254	+127	+126
⋮	⋮	⋮	⋮
10000001	129	+2	+1
10000000	128	+1	0
01111111	127	0	−1
01111110	126	−1	−2
⋮	⋮	⋮	⋮
00000001	1	−126	−127
00000000	0	−127	−128

3.6 부동소수점 수의 표현

P193~p204

■ 부동소수점 표현 예제

예제 3–27

10진수 $N = -13.625$에 대한 32-비트 부동소수점 표현을 구하라. 128 바이어스의 경우

풀이

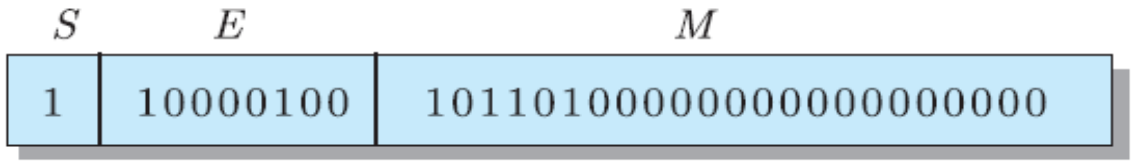
$13.625_{10} = 1101.101_2 = 0.1101101 \times 2^4$

부호(S) 비트 = 1 (−)

지수(E) = 00000100+10000000 = 10000100 (바이어스 128을 더한다)

가수(M) = 10110100000000000000000 (소수점 우측의 첫 번째 '1'은 제외)

<결과>



3.6 부동소수점 수의 표현

P193~p204

■ 부동소수점 수의 표현 범위

✓ 부동소수점 수의 표현 범위

➢ $0.5 \times 2^{-128} \sim (1 - 2^{-24}) \times 2^{127}$ 사이의 양수들

❖ 대략 $1.47 \times 10^{-39} \sim 1.7 \times 10^{38}$

➢ $-(1 - 2^{-24}) \times 2^{127} \sim -(0.5 \times 2^{-128})$ 사이의 음수들

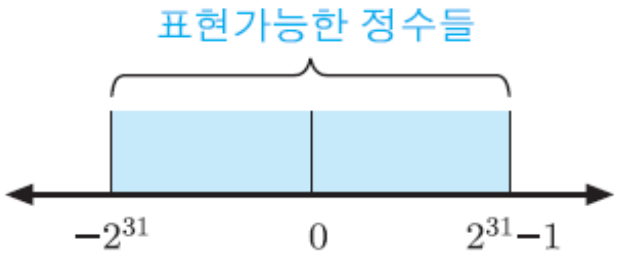
✓ 제외되는 범위

1) $(1 - 2^{-24}) \times 2^{127}$보다 작은 음수 → 음수 오버플로우(negative overflow)
2) 0.5×2^{-128} 보다 큰 음수 → 음수 언더플로우(negative underflow)
3) 0
4) 0.5×2^{-128} 보다 작은 양수 → 양수 언더플로우(positive underflow)
5) $(1 - 2^{-24}) \times 2^{127}$ 보다 큰 양수 → 양수 오버플로우(positive overflow)

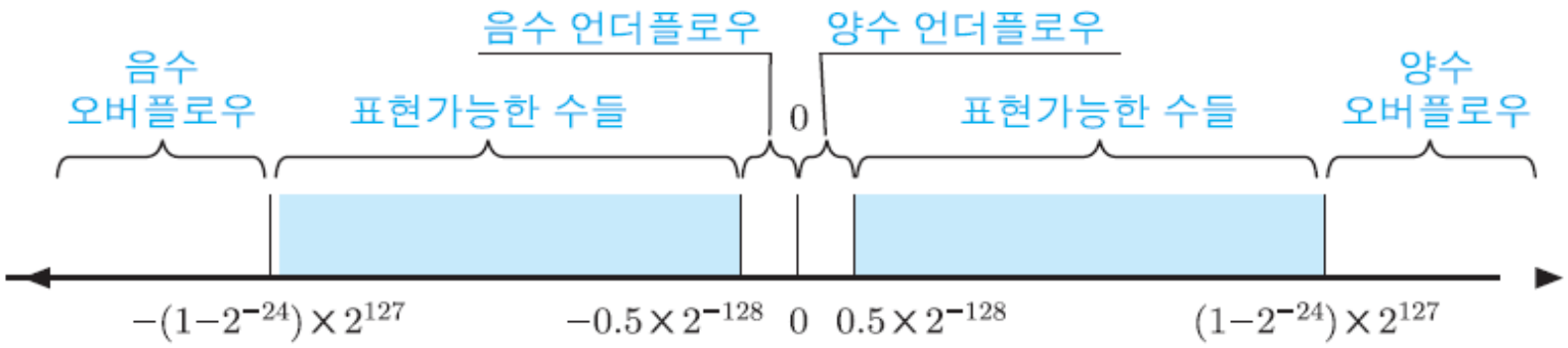
3.6 부동소수점 수의 표현

P193~p204

■ 32-비트 데이터 형식의 표현 가능한 수의 범위



(a) 2의 보수 정수의 표현 범위



(b) 부동소수점 수의 표현 범위

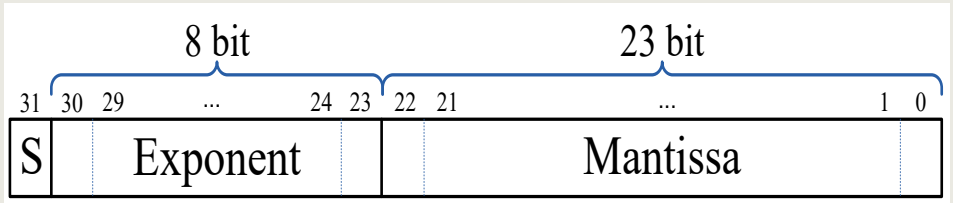
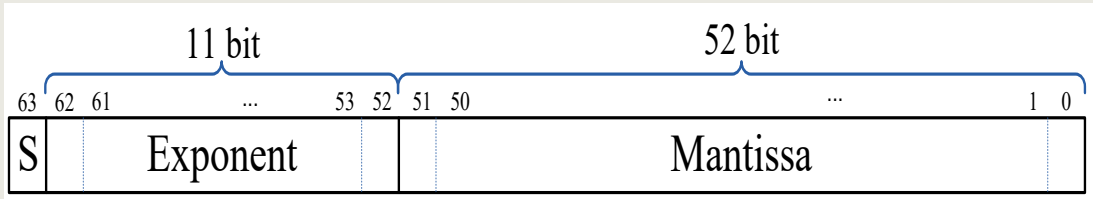
3.6 부동소수점 수의 표현

P193~p204

■ IEEE 754 표준 부동소수점 수의 형식→ 실제 컴퓨터에서 사용하는 표준

- ✓ 컴퓨터의 부동소수점수는 IEEE 754표준을 따른다.
- ✓ 부호(sign), 지수(exponent), 가수(mantissa)의 세 영역
  - ➢ ± **1.**bbbbb…bbb x 2^e
- ✓ 종류
  1) 단정도(single precision): $N = (-1)^S 2^{e-127}$ 히든 비트 : **1**.m
  2) 배정도(double precision): $N = (-1)^S 2^{e-1023}$ 히든 비트 : **1**.m

IEEE(Institute of Electrical and Electronics Engineers): 아이트리플이라고 발음. 전기/전자 및 컴퓨터 등의 공학분야에 관련된 표준을 개발하는 임무를 맡고 있는 전문기구. 전기/전자 및 컴퓨터 등의 공학 분야에 있어서 이 곳에서 발행하는 학술논문지가 세계최고의 권위를 인정받고 있다.

구분	IEEE 754 표준 부동소수점수의 비트 할당	바이어스	자릿수
단정도 부동소수점수		127	32비트
배정도 부동소수점수		1023	64비트

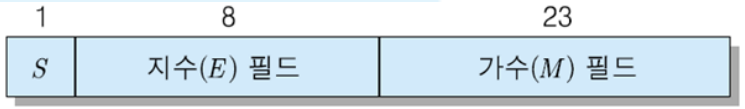
3.6 부동소수점 수의 표현

P193~p204

■ IEEE 754 표준 부동소수점 수의 형식 예제

예제 3–28

10진수 –13.625를 IEEE 754 단일-정밀도 표준 형식으로 표현하라.



(a) 단일–정밀도 형식(single–precision format)

풀이

$13.625_{10} = 1101.101_2 = 1.101101 \times 2^3$

부호(S) 비트 $= 1$ $(-)$

지수 $E = 00000011 + 01111111 = 10000010$ (바이어스 127을 더한다)

가수 $M = 10110100000000000000000$ (소수점 좌측의 1은 제외한다)

<결과>

S	E	M
1	10000010	10110100000000000000000

3.6 부동소수점 수의 표현

P193~p204

■ 예외(exception) 경우를 포함한 IEEE 754 표준

✓ 예외 경우를 포함한 정의 (32-비트 형식)

- 만약 E = 255이고 M ≠ 0이면, N = NaN
- 만약 E = 255이고 M = 0이면, N = $(-1)^S \infty$
- 만약 0 < E < 255 이면, N = $(-1)^S 2E^{-127}$ (1.M)
- 만약 E = 0이고 M ≠ 0이면, N = $(-1)^S 2^{-126}$ (0.M)
- 만약 E = 0이고 M = 0이면, N = $(-1)^S 0$

✓ 예외 경우를 포함한 정의 (64-비트 형식)

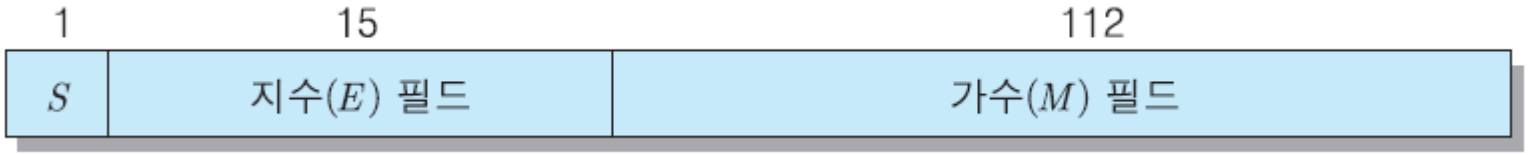
- 만약 E = 2047이고 M ≠ 0이면, N = NaN
- 만약 E = 2047이고 M = 0이면, N = $(-1)^S \infty$
- 만약 0 < E < 2047 이면, N = $(-1)^S 2E^{-1023}$ (1.M)
- 만약 E = 0이고 M ≠ 0이면, N = $(-1)^S 2^{-1022}$ (0.M)
- 만약 E = 0이고 M = 0이면, N = $(-1)^S 0$

3.6 부동소수점 수의 표현

P193~p204

■ IEEE 754-2008에서 추가된 정의

✓ 4배수-정밀도(quadruple-precision: 128-비트) 부동소수점 수의 표현



- **$N = (-1)^S 2E^{-16383} (1.M)$**
- 가수: 부호화-크기 표현 사용
- 지수 필드: 바이어스 16383 사용 (범위: -16383 ~ +16383)

3.6 부동소수점 수의 표현

P193~p204

■ IEEE 754 형식의 주요 파라미터들

표 3-5 IEEE 754 형식의 주요 파라미터들

파라미터	부동소수점 형식		
	단일 정밀도	복수 정밀도	4배수 정밀도
비트 수	32	64	128
지수 필드 폭(비트)	8	11	15
가수 필드 폭(비트)	23	52	112
지수 바이어스	127	1023	16383
최대 지수값	127	1023	16383
최소 지수값	−126	−1023	−16382
정수의 대략적 표현 범위 (최소/최대)	$\sim \pm 10^{-38} / \pm 10^{+38}$	$\sim \pm 10^{-308} / \pm 10^{+308}$	$\sim \pm 10^{-4932} / \pm 10^{+4932}$

3.7 부동소수점 산술 연산

P204~p209

■ 3.7.1 덧셈과 뺄셈

① 지수들이 일치되도록 조정 (alignment) → 소수점 위치의 일치

② 가수들 간의 연산(더하기 혹은 빼기) 수행

③ 결과를 정규화 (normalization)

✓ [참고] 10진 부동소수점 산술의 예

$$(135 \times 10^{-5}) + (246 \times 10^{-3}) \Rightarrow \begin{array}{r} 1.35 \times 10^{-3} \\ +\ 246 \quad \times 10^{-3} \\ \hline 247.35 \times 10^{-3} \end{array}$$

예제 3-29

부동소수점 수들 간의 덧셈 $(0.110100\times2^3+0.111100\times2^5)$을 수행하라.

풀이

이 덧셈은 아래와 같은 세 단계를 통하여 수행될 수 있다.

(1) 지수 조정

$$\begin{array}{r} 0.110100\times2^3 \\ +\ 0.111100\times2^5 \\ \hline \end{array} \rightarrow \begin{array}{r} 0.001101\times2^5 \\ +\ 0.111100\times2^5 \\ \hline 1.001001\times2^5 \end{array}$$

(2) 더하기

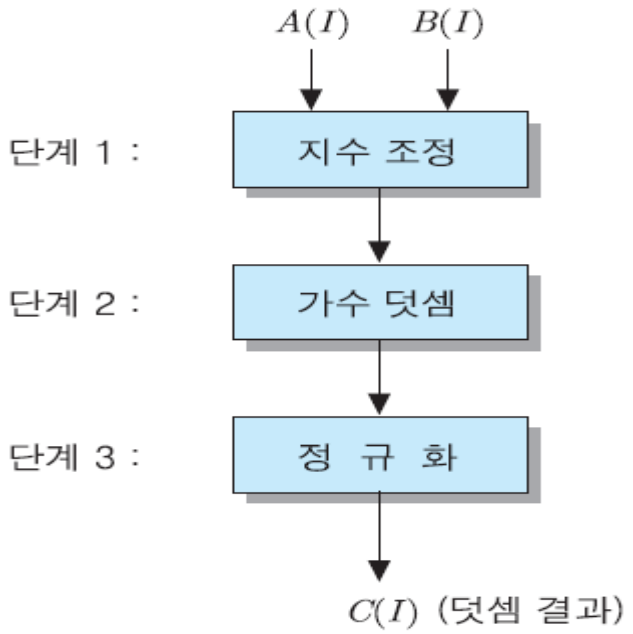
(3) 정규화 → 0.1001001×2^6 (최종 결과)

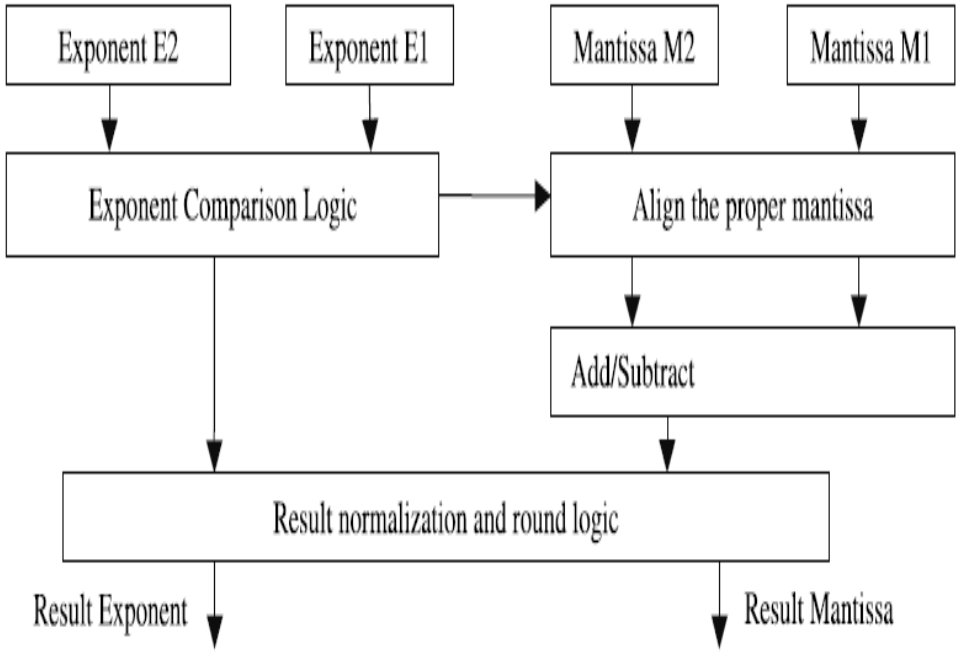
3.7 부동소수점 산술 연산

P204~p209

■ 부동소수점 산술의 파이프라이닝

- ✓ 연산 과정을 독립적 단계들로 분리 가능
- ✓ 단계 수만큼의 속도 향상
- ✓ 대규모의 부동소수점 계산을 처리하는 거의 모든 수퍼컴퓨터들에서 채택
- ✓ [예] 수 배열(number array)들 간의 덧셈
  - ➢ ***C(I) = A(I) + B(I)***





3.7 부동소수점 산술 연산

P204~p209

■ 3.7.2 부동소수점 곱셈 / 나눗셈

✓ 2진수 부동소수점 곱셈 과정

① 가수들을 곱한다

② 지수들을 더한다

③ 결과값을 정규화

✓ 2진수 부동소수점 나눗셈 과정

① 가수들을 나눈다

② 피제수의 지수에서 제수의 지수를 뺀다

③ 결과값을 정규화

[부동소수점 곱셈의 예]

$(0.1011 \times 2^3) \times (0.1001 \times 2^5)$

<가수 곱하기>

$1011 \times 1001 = 01100011$

<지수 더하기>

$3 + 5 = 8$

<정규화>

0.01100011×2^8

$= 0.1100011 \times 2^7$ (결과값)

3.7 부동소수점 산술 연산

■ 부동소수점의 산술 연산 – 곱셈/나눗셈 블록도

✓ 곱셈

① 유효숫자는 곱하고

② 지수는 더하고

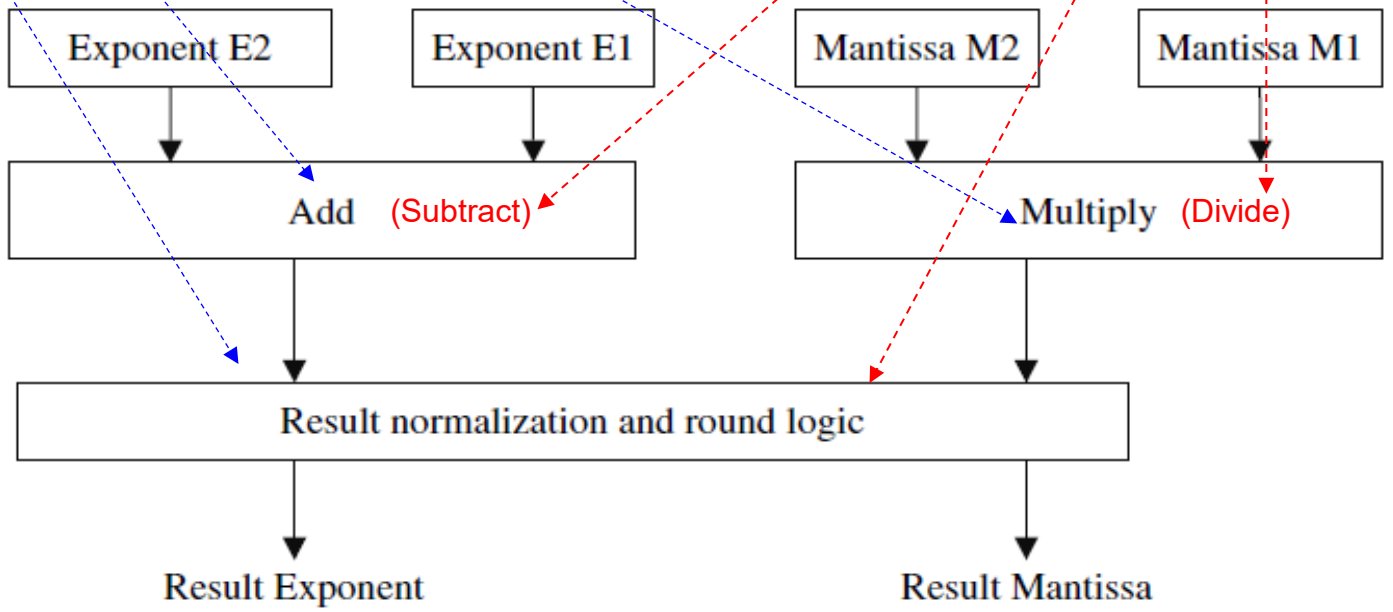
③ 정규화 실시

✓ 나눗셈

① 유효숫자는 나누고

② 지수는 빼고

③ 정규화 실시



3.7 부동소수점 산술 연산

P204~p209

■ 부동소수점 연산 과정에서 발생 가능한 문제점

- ✓ 지수 오버플로우(exponent overflow)
  - ➢ 양의 지수값이 최대 지수값을 초과
    - ❖ 수가 너무 커서 표현될 수 없는 상태이므로, +∞ 또는 -∞로 세트
- ✓ 지수 언더플로우(exponent underflow)
  - ➢ 음의 지수값이 최대 지수값을 초과
    - ❖ 수가 너무 작아서 표현될 수 없는 상태이므로, 0으로 세트
- ✓ 가수 언더플로우(mantissa underflow)
  - ➢ 가수의 소수점 위치 조정 과정에서 비트들이 가수의 우측 편으로 넘치는 상태
    - ❖ 반올림(rounding) 적용
- ✓ 가수 오버플로우(mantissa overflow)
  - ➢ 같은 부호를 가진 두 가수들을 덧셈하였을 때 올림수가 발생
    - ❖ 재조정(realignment) 과정을 통하여 정규화

Thank you